

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ

ΤΜΗΜΑ : ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ

ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ

ΤΟΜΕΑΣ: ΥΠΟΛΟΓΙΣΤΕΣ



Γραμμική και Συνδιαστική Βελτιστοποίηση

Εργασία #1

Όνομα:

Αικατερίνη Μαρία Παπαδάκη

Αριθμός Μητρώου:

1066566

Πάτρα, 11 Απριλίου 2025

Περιεχόμενα

1 Άσκηση 1	1
1.1 α	1
1.1.1 Κώδικας	1
1.1.2	4
1.1.3 Γράφημα 1	5
1.1.4 κώδικας για την βελτιστοποίηση	5
1.1.5 Γράφημα για εύρεση βέλτιστης λύσης	7
1.2 β	7
1.2.1 Κώδικας	7
1.2.2	8
1.2.3 Γράφημα	9
1.3 γ	9
1.3.1	9
1.3.2 κώδικας	9
1.3.3 Γραφήματα	11
2 Άσκηση 2	13
2.1 α	13
2.2 β	14
2.2.1 Γράφημα	16
2.2.2 Κώδικας	16

2.2.3 Αποτελέσματα	19
3 Άσκηση 3	20
4 Άσκηση 4	23
4.1 (a)	23
4.2 (b)	24
4.3 (c)	25
4.4 (d)	27
5 Άσκηση 5	28
5.1 Θεωρητικό Υπόβαθρο	28
5.2 α	30
5.2.1 Κώδικας	30
5.2.2 Αποτελέσματα	32
5.3 β	34
5.3.1 Κώδικας	35
5.3.2 Αποτελέσματα	37
5.4 γ	42
5.4.1 Κώδικας	42
5.4.2 Αποτελέσματα	43
6 Άσκηση 6	45
6.1 α	45
6.1.1 Κώδικας	46
6.1.2 Αποτελέσματα	48
6.2 β	49

1

Άσκηση 1

1.1 α

1.1.1 Κώδικας

```
import numpy as np
import matplotlib.pyplot as plt

# Άσκηση 1
#line a1x1+a2x2=b

def x2(x1, line):
    a1,a2,b = line
    if a2!=0:
        return (b-a1*x1)/a2
    else:
        return -1

# Εύρεση κοινών κορυφών δύο ευθειών της μορφής
# a11x1+a12x2 = b1
# a21x1+a22x2 = b2

def vertex(*hyperplanes):
    A = []
    b = []
    for h in hyperplanes:
        A.append(h[:-1])
        b.append(h[-1])
```

```
A = np.array(A)
b = np.array(b)
if np.linalg.det(A)!=0:
    v = tuple(np.linalg.solve(A, b))
    return v
else:
    return -1

# ορισμός ευθειών
x1 = np.linspace(-10, 10,500)

constraints_geq = ((6,3,12),
                   (4,8,16))
constraints_leq = ((6,5,30),
                   (6,7,36))
constraints_sign = ((1,0,0),
                   (0,1,0))

# sign constraints
min_x2 = np.zeros_like(x1)
min_x1 = np.zeros_like(x1)

x2_geq = [min_x2]
x2_leq = []
x1_geq = [min_x1]
x1_leq = []

for line in constraints_geq:
    x2_geq.append(x2(x1, line))

for line in constraints_leq:
    x2_leq.append(x2(x1,line))

where_condition = np.logical_and(
np.minimum.reduce(x2_leq)>= np.maximum.reduce(x2_geq),
x1 >= np.maximum.reduce(x1_geq)
)
```

```
plt.fill_between(x1,np.maximum.reduce(x2_geq), np.minimum.reduce(x2_leq), where=where_condition,
                 color='blue', alpha=0.3, label='Εφικτή Περιοχή')

plt.plot(x1, x2_geq[1], label='$6x_1 + 3x_2 = 12$')
plt.plot(x1, x2_geq[2], label='$4x_1 + 8x_2 = 16$')
plt.plot(x1, x2_leq[0], label='$6x_1 + 5x_2 = 30$')
plt.plot(x1, x2_leq[1], label='$6x_1 + 7x_2 = 36$')
plt.plot(x1, x2_geq[0], label='$x_1 = 0$')
plt.plot(x1_geq[0], x1, label='$x_2 = 0$')

vertexes_x=[]
vertexes_y=[]
for line1 in constraints_geq+constraints_leq+constraints_sign:
    for line2 in constraints_geq+constraints_leq+constraints_sign:
        v = vertex(line1,line2)
        if v!=-1:
            x, y = v
            #Κορυφές στην εφικτή περιοχή
            if (all(a * x + b * y >= c for a, b, c in constraints_geq+constraints_sign)
                and all(a * x + b * y <= c for a, b, c in constraints_leq)):

                vertexes_x.append(x)
                vertexes_y.append(y)

plt.scatter(vertexes_x, vertexes_y, color = 'red', zorder=5)

# Προσαρμογές γραφήματος
plt.xlim(-1, 7)
plt.ylim(-1, 7)

plt.xlabel(r'$x_1$')
plt.ylabel(r'$x_2$')
plt.legend()
plt.title("Εφικτή περιοχή")
# Εμφάνιση γραφήματος
```

```
plt.show()
```

1.1.2

Με τη συνάρτηση `x2(x1, line)` παίρνουμε την προβολή μιας ευθείας $line = a_1x_1 + a_2x_2$ στον άξονα x_2 .

Με τη συνάρτηση `vertexes` βρίσκουμε τις κοινές κορυφές των υπερεπιπέδων $H : a^T x = b$. Λύνοντας ένα σύστημα $Ax = b$. (Παραπάνω λεπτομέρειες για τα υπερεπίπεδα, εξηγούνται στην άσκηση 5.

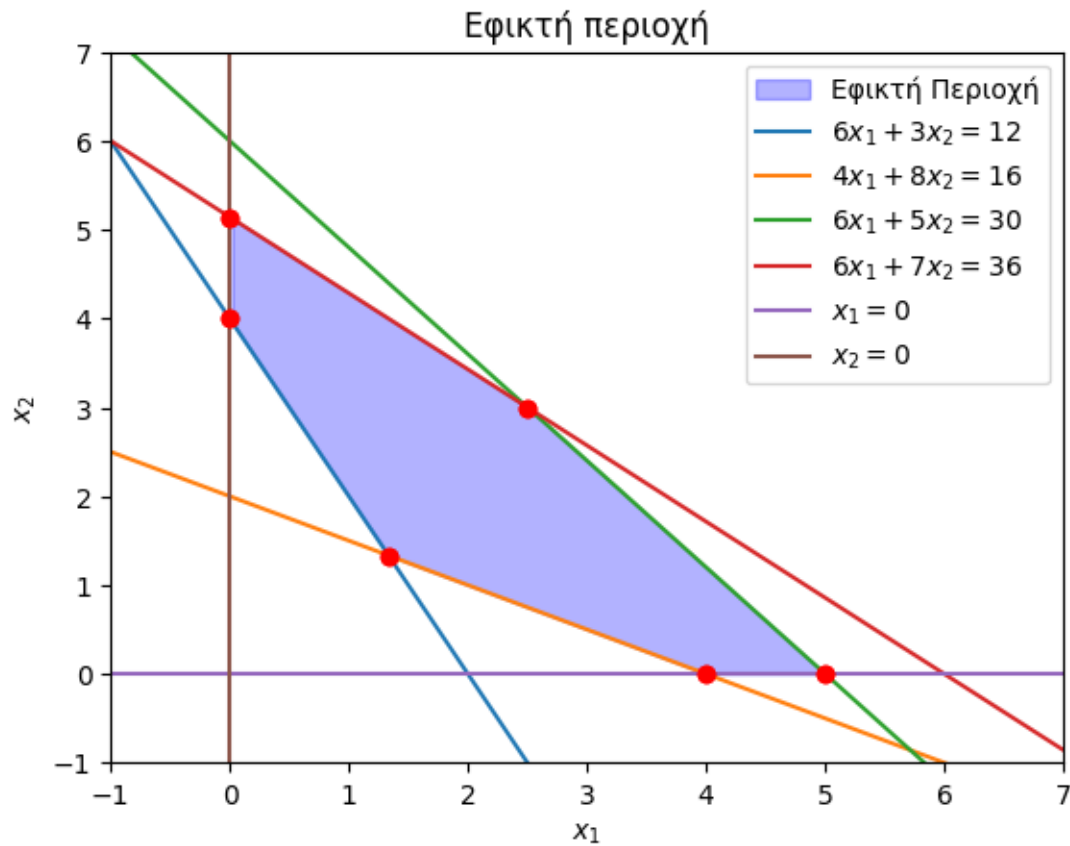
Έπειτα αρχικοποιούμε το x_1 , και τους περιορισμούς δίνοντας για κάθε περιορισμό ένα tuple με τους συντελεστές της γραμμής που θα όριζε ο κάθε περιορισμός αν ήταν ισότητα (a_1, a_2, b) . Τέλος υπολογίζουμε τα x_2 για κάθε γραμμή εξαρτημένα από το x_1 .

Θέλω για τους περιορισμούς $\leq$ και $\geq$ (συμπεριλαμβανομένου των περιορισμών προσήμου) να βρω τις ελάχιστες και τις μέγιστες τιμές για τα x_2 , αντίστοιχα. Και έτσι μπορώ να «γραμμοσκιάσω» την εφικτή περιοχή, που θα βρίσκεται ανάμεσα από τα μέγιστα και τα ελάχιστα x_2 , για τα θετικά x_1 .

Τέλος βρίσκω και τις κορυφές παίρνοντας όλες τους συνδιασμούς από τους περιορισμούς που όρισα παραπάνω και φτιάχνω μία λίστα με τα x και y από τις κορυφές.

Το αποτέλεσμα όλων των παραπάνω φαίνεται στο παρακάτω γράφημα:

1.1.3 Γράφημα 1



1.1.4 κώδικας για την βελτιστοποίηση

```
obj_values = np.linspace(0, 20, 5)
for z in obj_values:
    line=3,1,z
    if z==0:
        plt.plot(x1, x2(x1,line), '--', color = 'black', label='$Z = 3x_1 + x_2$', linewidth=0.5)
    else:
        plt.plot(x1, x2(x1,line), '--', color = 'black', linewidth=0.5)

where_condition = np.logical_and(np.minimum.reduce(x2_leq)>= np.maximum.reduce(x2_geq), x1 >= np.maximum.reduce(x1_geq))

plt.fill_between(x1,np.maximum.reduce(x2_geq), np.minimum.reduce(x2_leq), where=where_condition,
                color='blue', alpha=0.3, label='Εφικτή Περιοχή')

plt.plot(x1, x2_geq[1],linewidth=1, label='L1: $6x_1 + 3x_2 = 12$')
```

```
plt.plot(x1, x2_geq[2],linewidth=1, label='L2:  $4x_1 + 8x_2 = 16$ ')
plt.plot(x1, x2_leq[0],linewidth=1, label='L3:  $6x_1 + 5x_2 = 30$ ')
plt.plot(x1, x2_leq[1],linewidth=1, label='L4:  $6x_1 + 7x_2 = 36$ ')
plt.plot(x1, x2_geq[0],linewidth=1, label='L5:  $x_1 = 0$ ')
plt.plot(x1_geq[0], x1,linewidth=1, label='L6:  $x_2 = 0$ ')

vertexes_x=[]
vertexes_y=[]
for line1 in constraints_geq+constraints_leq+constraints_sign:
    for line2 in constraints_geq+constraints_leq+constraints_sign:
        v = vertex(line1,line2)
        if v!=-1:
            x, y = v
            #Κορυφές στην εφικτή περιοχή
            if (all(a * x + b * y >= c for a, b, c in constraints_geq+constraints_sign)
                and all(a * x + b * y <= c for a, b, c in constraints_leq)):

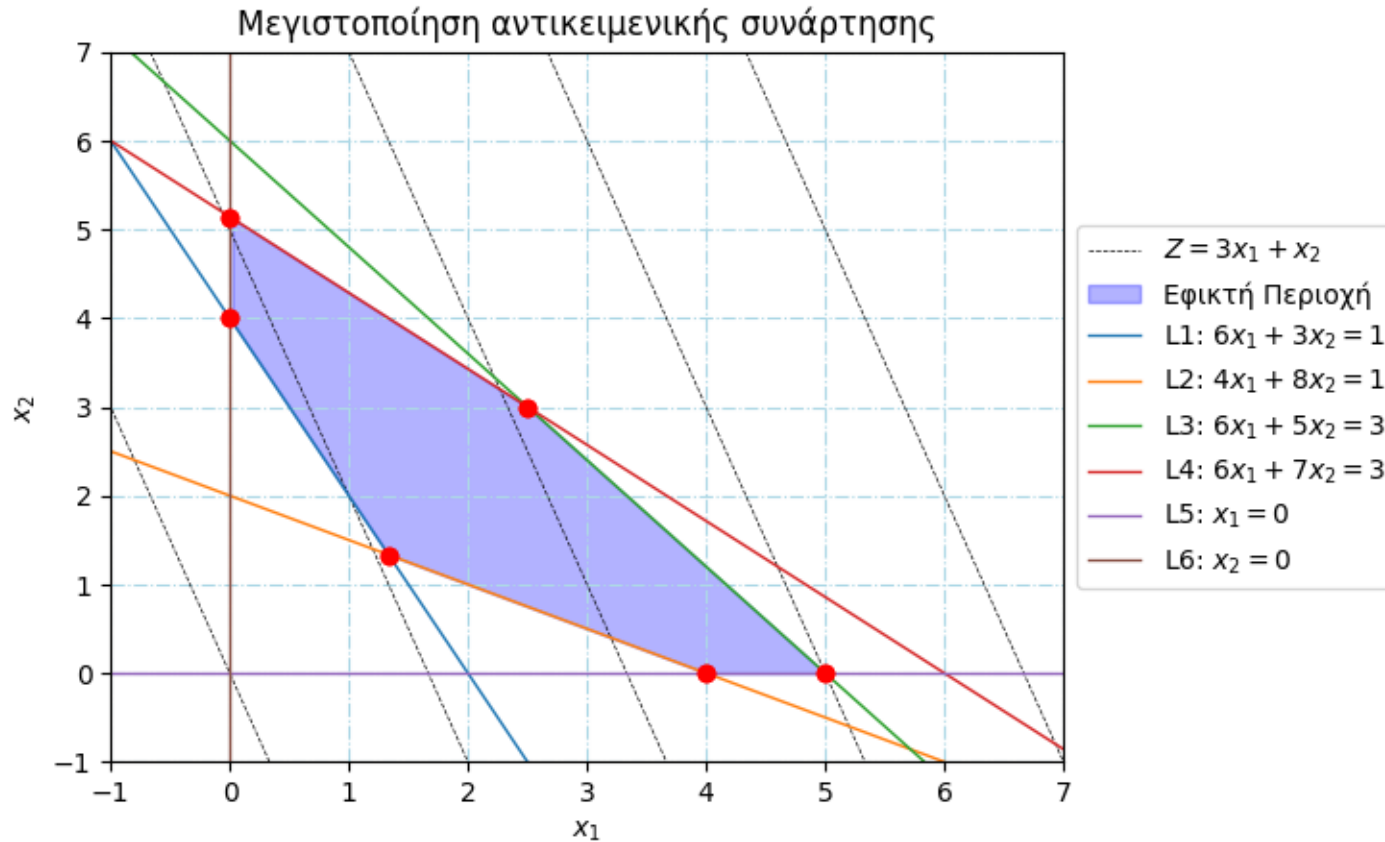
                vertexes_x.append(x)
                vertexes_y.append(y)

plt.scatter(vertexes_x, vertexes_y, color = 'red', zorder=5)
# Προσαρμογές γραφήματος
plt.xlim(-1, 7)
plt.ylim(-1, 7)

plt.xlabel(r' $x_1$ ')
plt.ylabel(r' $x_2$ ')
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5), fontsize=10)
plt.grid(True, linestyle='-.', color = 'lightblue')
plt.title("Μεγιστοποίηση αντικειμενικής συνάρτησης")
# Εμφάνιση γραφήματος
plt.show()
```

Παραπάνω κάνουμε ό,τι και πριν, ορίζοντας και τιμές για την αντικειμενική συνάρτηση 5 τιμές από 0 έως και 20. Για κάθε τιμή ορίζεται μία ευθεία και βρίσκουμε την προβολή της στον κάθετο άξονα όμοια με πριν.

1.1.5 Γράφημα για εύρεση βέλτιστης λύσης



Από ό,τι βλέπουμε από το γράφημα, μέγιστη τιμή της αντικειμενικής συνάρτησης είναι στην κορυφή $(5,0)$ και η τιμή της είναι $Z = 3 \cdot 5 + 0 = 15$

1.2 β

1.2.1 Κώδικας

```
x0,y0 = vertex(constraints_geq[0], constraints_geq[1])
c2 = [0.3,0.5,1,2,3]
for c in c2:
    line = (1,c,0)
    plots = plt.plot(x1+x0, x2(x1,line)+y0, label='$Z = x_1 + {:.2f}x_2$'.format(c), linewidth=1)
    plt.plot(x1+x0-0.1, x2(x1,line)+y0-0.1, '--', color = plots[0].get_color(), linewidth=1)

plt.fill_between(x1,np.maximum.reduce(x2_geq), np.minimum.reduce(x2_leq), where=where_condition,
                color='blue', alpha=0.3, label='Εφικτή Περιοχή')
```

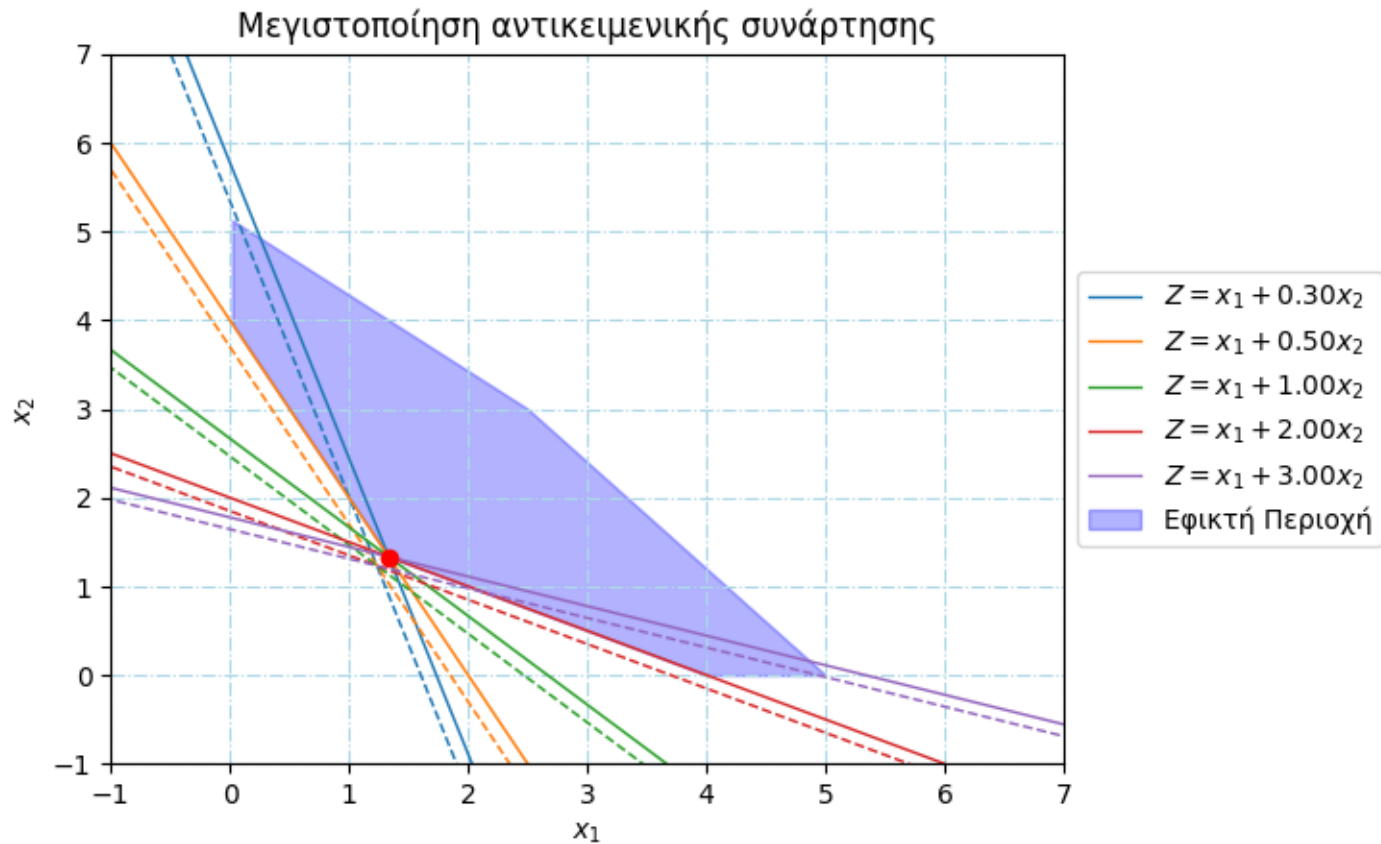
```
plt.scatter(x0, y0, color = 'red', zorder=5)
# Προσαρμογές γραφήματος
plt.xlim(-1, 7)
plt.ylim(-1, 7)

plt.xlabel(r'$x_1$')
plt.ylabel(r'$x_2$')
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5), fontsize=10)
plt.grid(True, linestyle='-.', color = 'lightblue')
plt.title("Μεγιστοποίηση αντικειμενικής συνάρτησης")
# Εμφάνιση γραφήματος
plt.show()
```

1.2.2

Ξανακάνουμε ότι κάναμε και με την παραπάνω συνάρτηση στον κώδικα μας, αλλά επικεντρωνόμαστε αυτήν την φορά μόνο στην κορυφή των ευθειών που ορίζουν οι Π_1, Π_2 . Όπως Φαίνεται και στο παρακάτω γράφημα η κλίση της ευθείας της αντικειμενικής συνάρτησης πρέπει να είναι μεταξύ των κλίσεων Π_1, Π_2 . Αλλιώς για λίγο μικρότερο Z , θα βρισκόμασταν ακόμα στην εφικτή περιοχή. Όπως βλέπουμε για το c_2 ισχύει: $0.5 < c_2 < 2$ (Την ισότητα την παραλείπουμε, γιατί δεν θα ήταν η μοναδική βέλτιστη λύση η κορυφή που μας ενδιαφέρει, αλλά οι βέλτιστες λύσεις θα βρίσκονταν σε ένα ευθύγραμμο τμήμα. Εξάλλου οι κλίσεις της Π_1 και της Π_2 είναι $\frac{3}{6} = 0.5$ και $\frac{8}{4} = 2$ αντίστοιχα.

1.2.3 Γράφημα



1.3 γ

1.3.1

Οι κλίσεις των Π3 και Π4 είναι $\frac{5}{6}$ και $\frac{7}{6} = 2$ αντίστοιχα. Συνεπώς $\frac{5}{6} < \frac{c_1}{c_2} < \frac{7}{6}$. Τα c_1, c_2 είναι ομόσημα. Συνεπώς:

$$\begin{aligned} \frac{7}{6}c_2 < c_1 < \frac{5}{6}c_2 & , \quad c_1, c_2 > 0 \\ \frac{5}{6}c_2 < c_1 < \frac{7}{6}c_2 & , \quad c_1, c_2 < 0 \end{aligned}$$

1.3.2 κώδικας

Έτσι προσαρμόστηκε και ο κώδικας ως εξής (παίρνοντας τη περίπτωση των θετικών c , που πρακτικά, θα βλέπαμε κάτι αντίστοιχο και στην περίπτωση των αρνητικών c .)

```
x0,y0 = vertex(constraints_leq[0], constraints_leq[1])

c1 = np.array([5,6,7])
c2 = np.linspace(4,8,5)

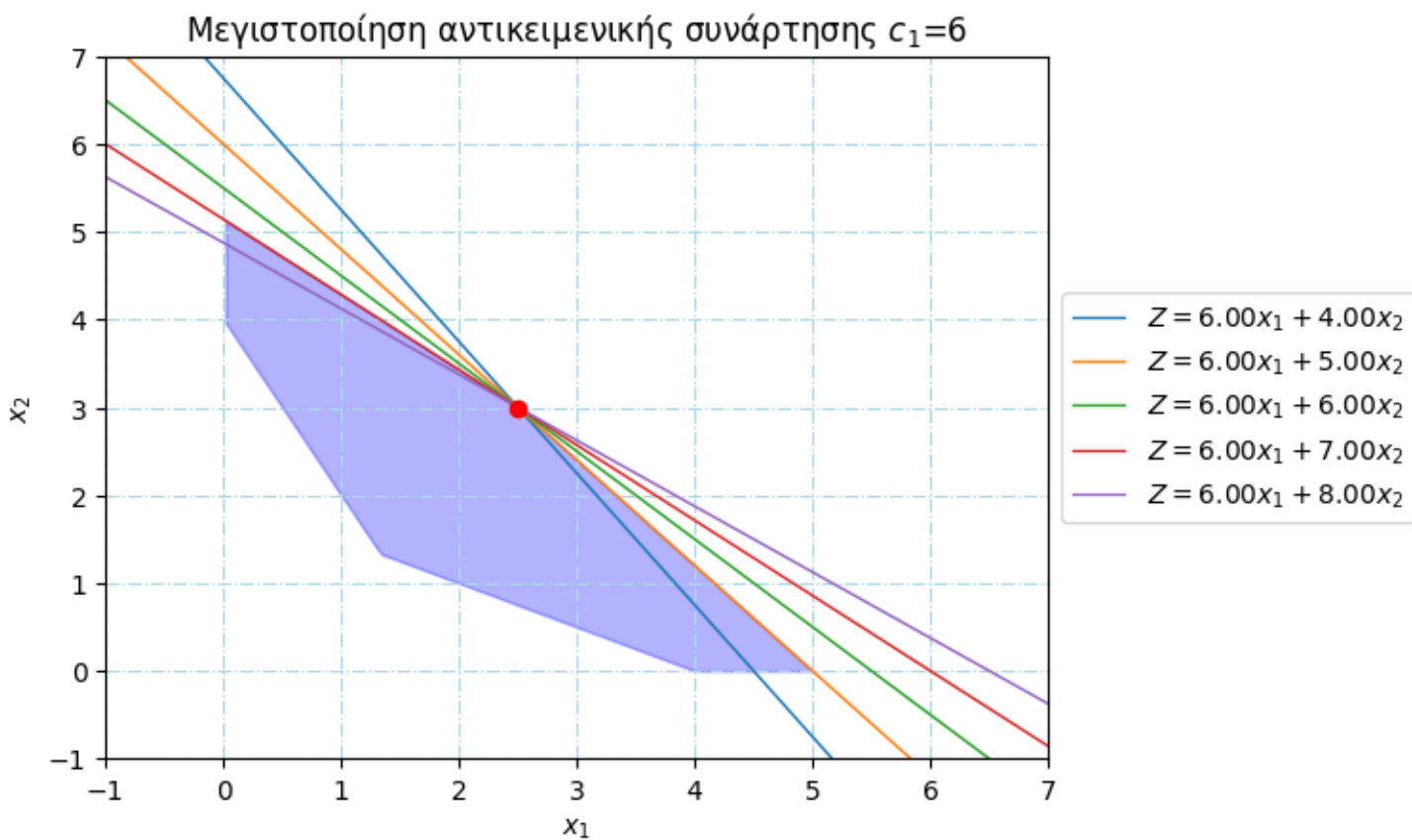
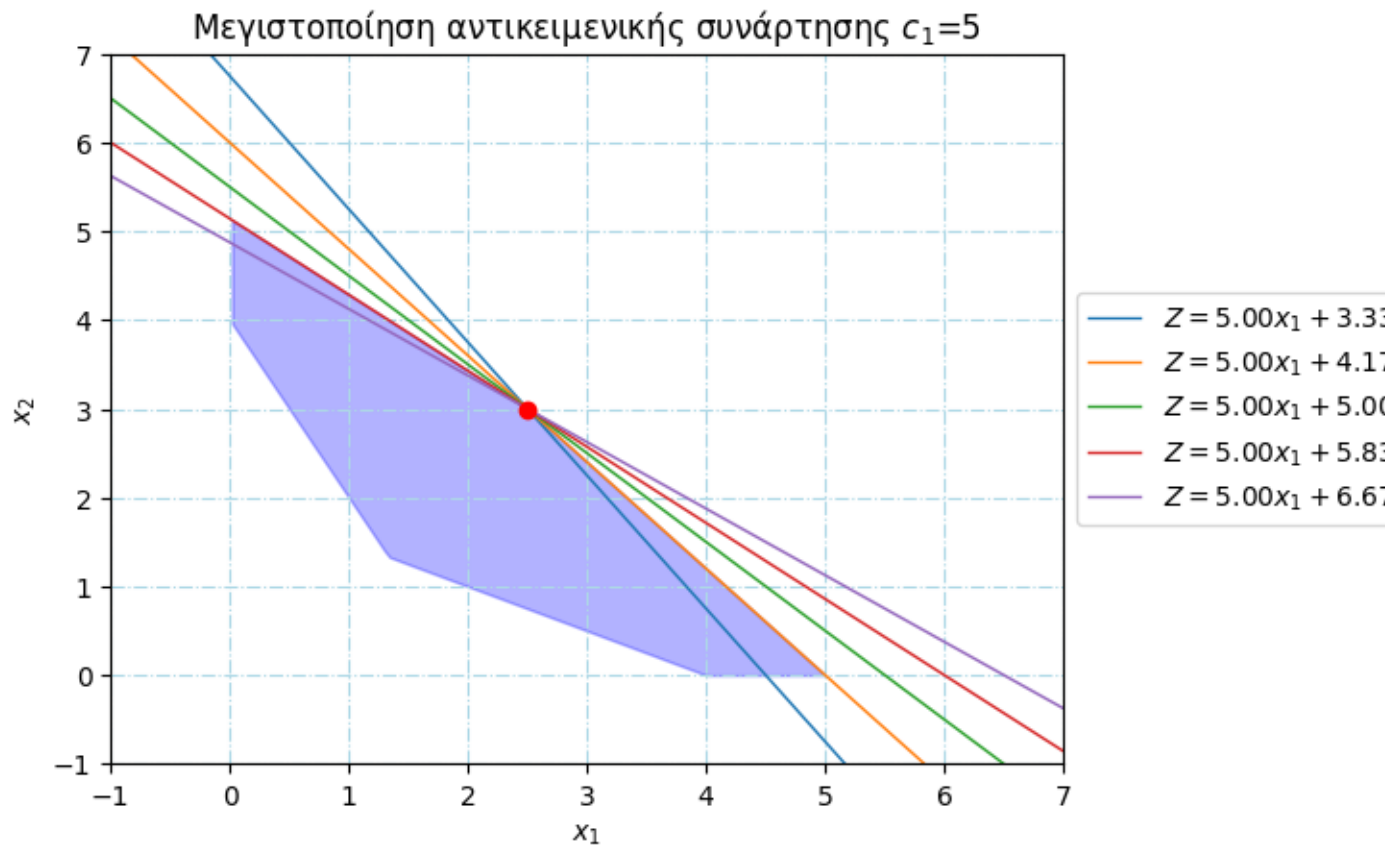
for c1p in c1:
    c2p=c2*c1p/6
    for c2pp in c2p:
        line = (c1p,c2pp, 0)
        plt.plot(x1+x0, x2(x1,line)+y0, label='$Z = {:.2f}x_1 - {:.2f}x_2$'.format(c1p,c2pp), linewidth=1)

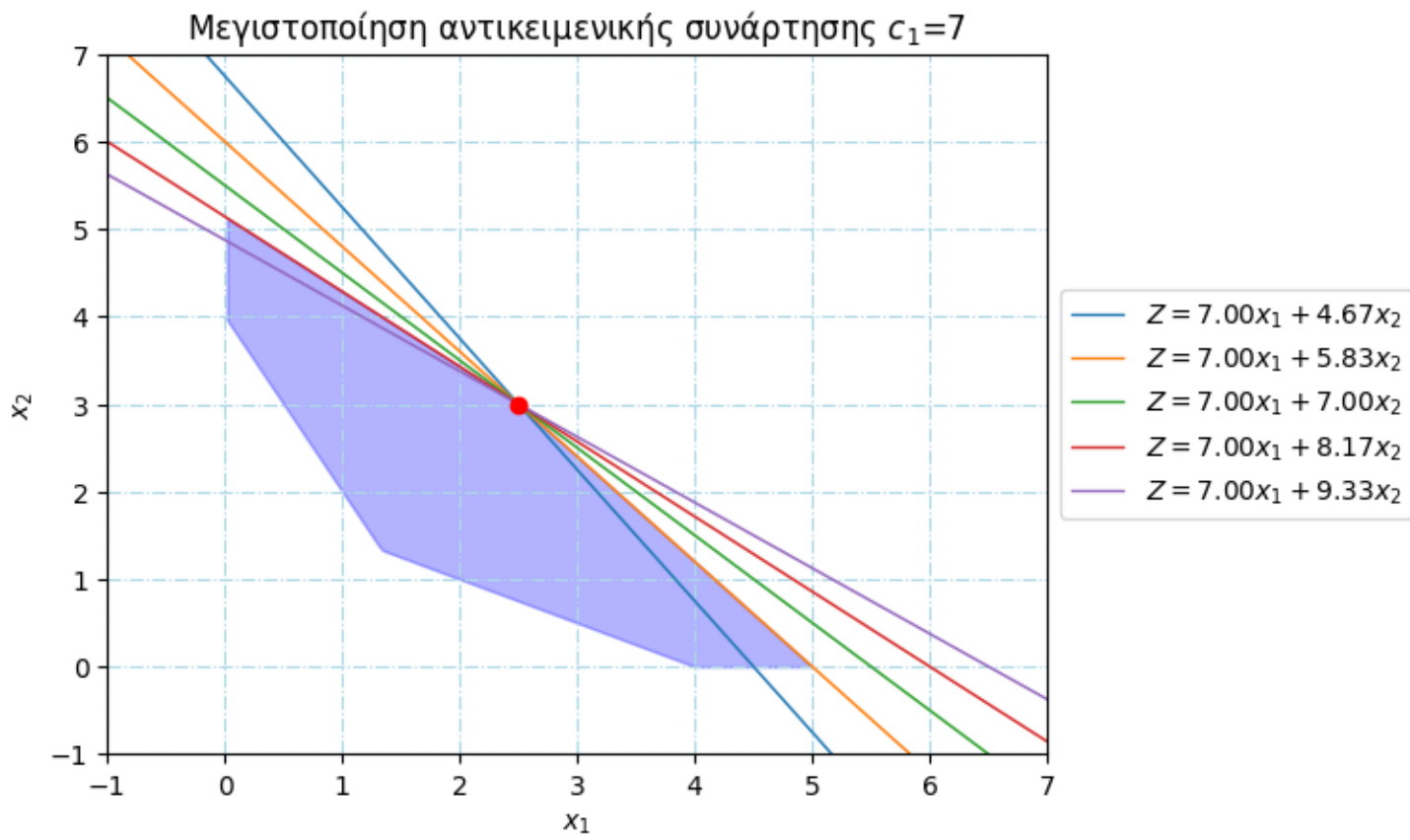
plt.fill_between(x1,np.maximum.reduce(x2_geq), np.minimum.reduce(x2_leq), where=where_condition,
                color='blue', alpha=0.3)#, label='Εφικτή Περιοχή')

plt.scatter(x0, y0, color = 'red', zorder=5)
# Προσαρμογές γραφήματος
plt.xlim(-1, 7)
plt.ylim(-1, 7)

plt.xlabel(r'$x_1$')
plt.ylabel(r'$x_2$')
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5), fontsize=10)
plt.grid(True, linestyle='-.', color = 'lightblue')
plt.title("Μεγιστοποίηση αντικειμενικής συνάρτησης $c_1$={}".format(c1p))
# Εμφάνιση γραφήματος
plt.show()
```

1.3.3 Γραφήματα





2

Άσκηση 2

2.1 α

- Μεταβλητές απόφασης: Στο παρόν πρόβλημα, καλούμαστε να υπολογίσουμε την ένταση της ακτινοβολίας της κάθε δέσμης. Συνεπώς **οι εντάσεις της κάθε δέσμης** θα αποτελέσουν τις δύο μεταβλητές απόφασης x_1, x_2 .
- Περιορισμοί: Από τον πίνακα των περιορισμών που δίνονται μπορούν εύκολα να προκύψουν οι παρακάτω εξισώσεις
$$0.3x_1 + 0.1x_2 \leq 2.7$$
$$0.5x_1 + 0.5x_2 = 6$$
$$0.6x_1 + 0.4x_2 \geq 6$$
- Περιορισμοί προσήμου: Προφανώς η ένταση της ακτινοβολίας δεν μπορεί να έχει αρνητική τιμή: $x_1, x_2 \geq 0$
- Αντικειμενική Συνάρτηση: Η ακτινοβολία θα πρέπει να «χτυπήσει» κατά τον μικρότερο δυνατό τρόπο την υγιή περιοχή και συνεπώς $Z = 0.4x_1 + 0.5x_2$, που προσδιορίζει την **ένταση της ακτινοβολίας στην υγιή περιοχή**.

Πρέπει λοιπόν τα x_1, x_2 , να υπολογιστούν κατάλληλα, ούτως ώστε να ικανοποιούν και τους παραπάνω περιορισμούς, αλλά και να ελαχιστοποιούν την αντικειμενική συνάρτηση Z . Δηλαδή $(x_1^0, x_2^0) = \arg \min_{(x_1, x_2)} Z$, για x_1, x_2 επιλεγμένα έτσι ώστε να ικανοποιούν όλους τους περιορισμούς.

2.2 β

```
# ορισμός ευθειών
```

```
x1 = np.linspace(-10, 50, 100000)
```

```
constraints_leq2 = [(0.3, 0.1, 2.7)]
```

```
constraints_eq2 = [(0.5, 0.5, 6)]
```

```
constraints_geq2 = [(0.6, 0.4, 6)]
```

```
constraints_sign2 = [(1, 0, 0),  
                     (0, 1, 0)]
```

```
# sign constraints
```

```
min_x2 = np.zeros_like(x1)
```

```
min_x1 = np.zeros_like(x1)
```

```
x2_geq2 = [min_x2]
```

```
x2_leq2 = []
```

```
x1_geq2 = [min_x1]
```

```
x1_leq2 = []
```

```
x2_eq2 = []
```

```
for line in constraints_geq2:  
    x2_geq2.append(x2(x1, line))
```

```
for line in constraints_leq2:  
    x2_leq2.append(x2(x1, line))
```

```
for line in constraints_eq2:  
    x2_eq2.append(x2(x1, line))
```

```
where_condition2 = np.logical_and(np.minimum.reduce(x2_leq2) >= np.maximum.reduce(x2_geq2), x1 >= np.max
```

```
plt.fill_between(x1, np.maximum.reduce(x2_geq2), np.minimum.reduce(x2_leq2), where=where_condition2,  
                 color='blue', alpha=0.3, label='Περιορισμοί από ανισώσεις')
```

```
plt.plot(x1, x2_leq2[0], label='$0.3x_1 + 0.1x_2 = 2.7$')
```

```
plt.plot(x1, x2_eq2[0], label='$0.5x_1 + 0.5x_2 = 6$')
```

```
plt.plot(x1, x2_geq2[1], label='$0.6x_1 + 0.4x_2 = 12$')
```

```
plt.plot(x1, x2_geq2[0], label='$x_1 = 0$')
```

```
plt.plot(x1_geq2[0], x1, label='$x_2 = 0$')

vertexes_x=[]
vertexes_y=[]

for line1 in constraints_geq2+constraints_leq2+constraints_sign2+constraints_eq2:
    for line2 in constraints_geq2+constraints_leq2+constraints_sign2+constraints_eq2:
        v = vertex(line1,line2)
        if v!=-1:
            x, y = v
            x=round(x,2)
            y=round(y,2)
            #Κορυφές στην εφικτή περιοχή
            if (all(a * x + b * y >= c for a, b, c in constraints_geq2+constraints_sign2)
                and all(a * x + b * y <= c for a, b, c in constraints_leq2)
                and all(a * x + b * y == c for a, b, c in constraints_eq2)
                and not (x in vertexes_x and y in vertexes_y)):

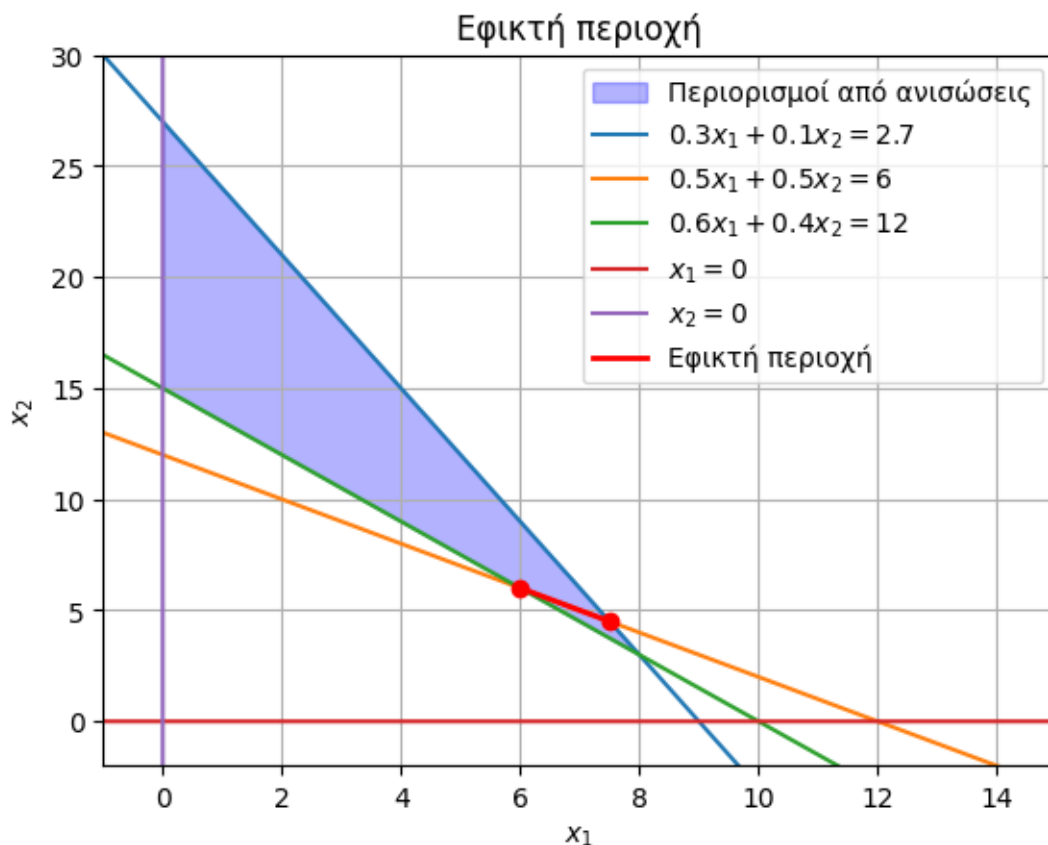
                vertexes_x.append(x)
                vertexes_y.append(y)

plt.scatter(vertexes_x, vertexes_y, color = 'red', zorder=5)
plt.plot(vertexes_x, vertexes_y, 'red', linewidth=2, label="Εφικτή περιοχή")
for i in range(len(vertexes_x)):
    print(vertexes_x[i], vertexes_y[i])

# Προσαρμογές γραφήματος
plt.xlim(-1, 15)
plt.ylim(-2, 30)
plt.xlabel(r'$x_1$')
plt.ylabel(r'$x_2$')
plt.legend()
plt.title("Εφικτή περιοχή")
plt.grid(True)
# Εμφάνιση γραφήματος
plt.show()
```

Κάνουμε ό,τι ακριβώς κάναμε και στο πρόβλημα 1, απλά για τους περιορισμούς του παρόντος προβλήματος. Εδώ, ορίσαμε και περιορισμούς ισότητας, οπότε φτιάξαμε την εφικτή περιοχή, που δημιουργούν οι ανισώσεις, αλλά πήραμε το ευθύγραμμο τμήμα που βρίσκεται σε αυτήν την περιοχή για να βρούμε την τελική εφικτή περιοχή.

2.2.1 Γράφημα



Έπειτα όπως και στην άσκηση 1 παίρνουμε διάφορα Z και τυπώνουμε τις παράλληλες ευθείες που ορίζονται από την αντικειμενικής συνάρτηση. Παρακάτω είναι ο κώδικας και τα αποτελέσματα.

2.2.2 Κώδικας

```
obj_values = np.linspace(5.175,5.475,5)
print(obj_values)
for z in obj_values:
    line=0.4,0.5,z
    if z==5.175:
```

```

plt.plot(x1, x2(x1,line), '--', color = 'black', label='$Z = 0.4x_1 + 0.5x_2$', linewidth=0.5)
else:
    plt.plot(x1, x2(x1,line), '--', color = 'black', linewidth=0.5)

# ορισμός ευθειών
x1 = np.linspace(-10, 50,100000)

constraints_leq2 = [(0.3,0.1,2.7)]
constraints_eq2 = [(0.5,0.5,6)]
constraints_geq2 = [(0.6,0.4,6)]
constraints_sign2 = [(1,0,0),
                     (0,1,0)]

# sign constraints
min_x2 = np.zeros_like(x1)
min_x1 = np.zeros_like(x1)

x2_geq2 = [min_x2]
x2_leq2 = []
x1_geq2 = [min_x1]
x1_leq2 = []
x2_eq2 = []

for line in constraints_geq2:
    x2_geq2.append(x2(x1, line))

for line in constraints_leq2:
    x2_leq2.append(x2(x1,line))

for line in constraints_eq2:
    x2_eq2.append(x2(x1,line))

where_condition2 = np.logical_and(np.minimum.reduce(x2_leq2)>= np.maximum.reduce(x2_geq2), x1 >= np.max

plt.fill_between(x1,np.maximum.reduce(x2_geq2), np.minimum.reduce(x2_leq2), where=where_condition2,
                color='blue', alpha=0.3, label='Περιορισμοί από ανισώσεις')

plt.plot(x1, x2_leq2[0], label='$0.3x_1 + 0.1x_2 = 2.7$')
plt.plot(x1, x2_eq2[0], label='$0.5x_1 + 0.5x_2 = 6$')
plt.plot(x1, x2_geq2[1], label='$0.6x_1 + 0.4x_2 = 12$')

```

```

plt.plot(x1, x2_geq2[0], label='$x_1 = 0$')
plt.plot(x1_geq2[0], x1, label='$x_2 = 0$')

vertexes_x=[]
vertexes_y=[]

for line1 in constraints_geq2+constraints_leq2+constraints_sign2+constraints_eq2:
    for line2 in constraints_geq2+constraints_leq2+constraints_sign2+constraints_eq2:
        v = vertex(line1,line2)
        if v!=-1:
            x, y = v
            x=round(x,2)
            y=round(y,2)
            #Κορυφές στην εφικτή περιοχή
            if (all(a * x + b * y >= c for a, b, c in constraints_geq2+constraints_sign2)
                and all(a * x + b * y <= c for a, b, c in constraints_leq2)
                and all(a * x + b * y == c for a, b, c in constraints_eq2)
                and not (x in vertexes_x and y in vertexes_y)):

                vertexes_x.append(x)
                vertexes_y.append(y)

plt.scatter(vertexes_x, vertexes_y, color = 'red', zorder=5)
plt.plot(vertexes_x, vertexes_y, 'red', linewidth=2, label="Εφικτή περιοχή")
for i in range(len(vertexes_x)):
    print(vertexes_x[i], vertexes_y[i])

# Προσαρμογές γραφήματος
plt.xlim(5, 8)
plt.ylim(4, 7)
plt.xlabel(r'$x_1$')
plt.ylabel(r'$x_2$')
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5), fontsize=10)
plt.title("Ελαχιστοποίηση αντικειμενικής συνάρτησης")
plt.grid(True)
# Εμφάνιση γραφήματος
plt.show()

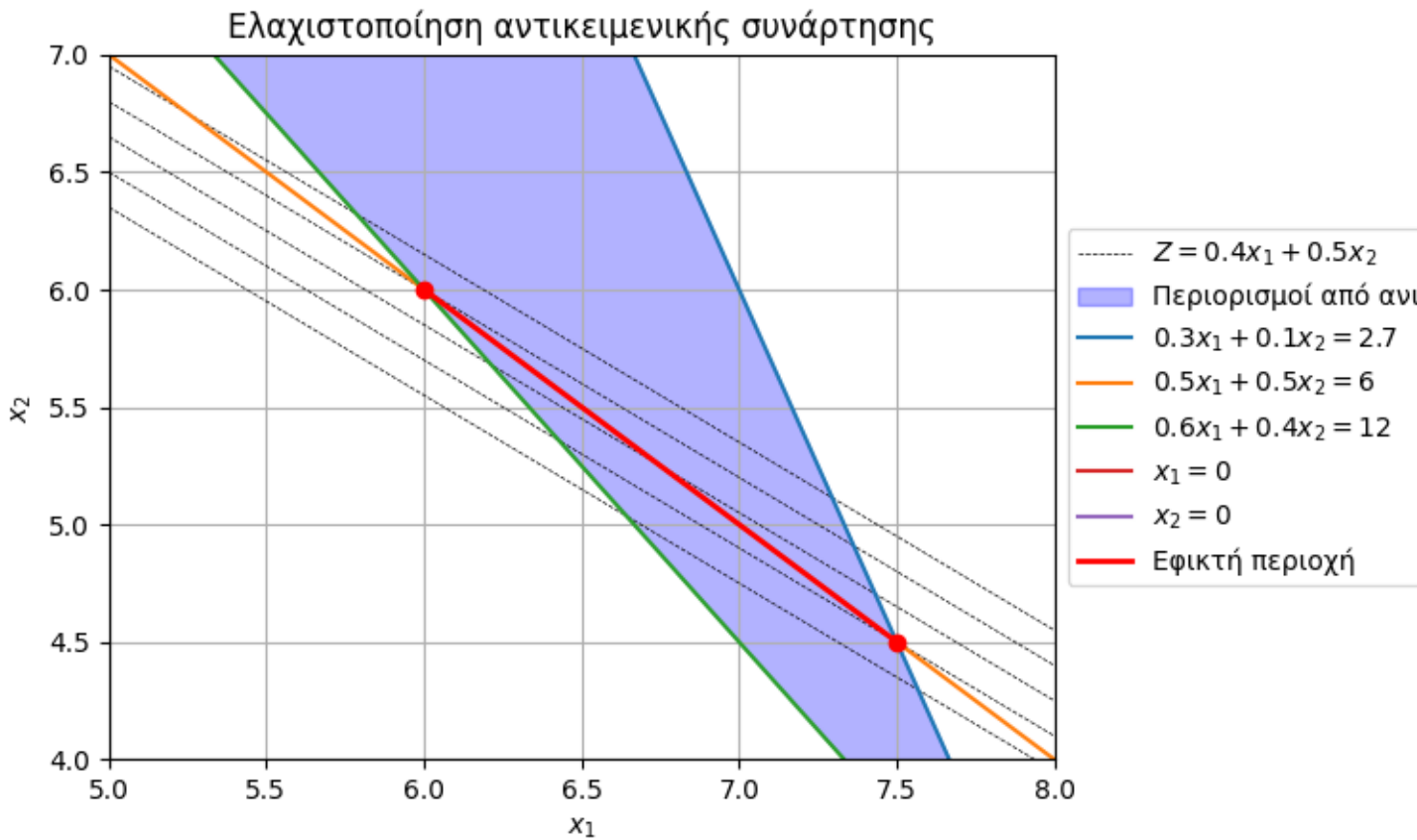
```

2.2.3 Αποτελέσματα

[5.175 5.25 5.325 5.4 5.475]

6.0 6.0

7.5 4.5



Βλέπουμε λοιπόν τις διάφορες τιμές της αντικειμενικής συνάρτησης και τις δύο κορυφές. Βλέπουμε ότι ελαχιστοποιείται η αντικειμενική συνάρτηση στο σημείο $(7.5, 4.5)$ και η αντικειμενική συνάρτηση: $Z = 0.4 \cdot 7.5 + 0.5 \cdot 4.5 = 3 + 2.25 = 5.25$

3

Άσκηση 3

- Μεταβλητές απόφασης: Στο παρόν πρόβλημα, η επιχείρηση καλείται να αξιοποιήσει τις πρώτες ύλες τις με σκοπό να παράξει ζωοτροφές, με όσο το δυνατόν χαμηλότερο κόστος. Συνεπώς, **οι ποσότητες πρώτων υλών (σε τόνους)** θα αποτελέσουν τις τέσσερις μεταβλητές απόφασης για κάθε τύπο ζωοτροφής. Έτσι θα έχουμε σύνολο 12 μεταβλητές απόφασης x_{ij} όπου i ο τύπος τις ζωοτροφής $i \in \{1, 2, 3\}$ και j η πρώτη ύλη $j \in \{1, 2, 3, 4\}$. $x_{11}, x_{12}, x_{13}, x_{14}, x_{21}, x_{22}, x_{23}, x_{24}, x_{31}, x_{32}, x_{33}, x_{34}$.
- Περιορισμοί: Παράγονται σύνολο 12,8,9 τόννοι ζωοτροφής για κάθε τύπο ζωοτροφής.

Ζωοτροφή	Περιορισμοί από παραγωγή
τύπου I	$x_{11} + x_{12} + x_{13} + x_{14} = 12$
τύπου II	$x_{21} + x_{22} + x_{23} + x_{24} = 8$
τύπου III	$x_{31} + x_{32} + x_{33} + x_{34} = 9$

Στη διάθεση μας έχουμε 9 τόνους καλαμπόκι, 12 ασβεστόλιθο, 5 σόγια, 6 ιχθυάλευρα. Έτσι προκύπτουν οι παρακάτω περιορισμοί

Πρώτη ύλη	Περιορισμοί από ποσότητες πρώτων υλών
Καλαμπόκι	$x_{11} + x_{21} + x_{31} \leq 9$
Ασβεστόλιθος	$x_{12} + x_{22} + x_{32} \leq 12$
Σόγια	$x_{13} + x_{23} + x_{33} \leq 5$
Ιχθυάλευρο	$x_{14} + x_{24} + x_{34} \leq 6$

Αξιοποιούμε τον Πίνακα 1 του προβλήματος, για να υπολογίσουμε τις ποσότητες των θρεπτικών συστατικών δεδομένων των ποσοτήτων πρώτων υλών. Αξιοποιούμε τον Πίνακα 2 του προβλήματος, για να ορίσουμε άνω και κάτω φράγματα για τις ποσότητες των θρεπτικών συστατικών, σύμφωνα με τις ανάγκες της κάθε ζωοτροφής. Δεδομένου όμως ότι έχουμε μία συγκεκριμένη μάζα παραγωγής από κάθε ζωοτροφή πολλαπλασιάζουμε τα min και max του πίνακα με την μάζα παραγωγής κάθε ζωοτροφής.

Τύπου I	Βιταμίνες:	$8x_{11} + 6x_{12} + 10x_{13} + 4x_{14} \geq 72 = 6 \cdot 12$
	Πρωτεΐνες:	$10x_{11} + 5x_{12} + 12x_{13} + 8x_{14} \geq 72 = 6 \cdot 12$
	Ασβέστιο:	$6x_{11} + 10x_{12} + 6x_{13} + 6x_{14} \geq 84 = 7 \cdot 12$
	Λίπος:	$8x_{11} + 6x_{12} + 6x_{13} + 9x_{14} \geq 48 = 4 \cdot 12$
		$8x_{11} + 6x_{12} + 6x_{13} + 9x_{14} \leq 96 = 8 \cdot 12$
Τύπου II	Βιταμίνες:	$8x_{21} + 6x_{22} + 10x_{23} + 4x_{24} \geq 48 = 6 \cdot 8$
	Πρωτεΐνες:	$10x_{21} + 5x_{22} + 12x_{23} + 8x_{24} \geq 48 = 6 \cdot 8$
	Ασβέστιο:	$6x_{21} + 10x_{22} + 6x_{23} + 6x_{24} \geq 48 = 6 \cdot 8$
	Λίπος:	$8x_{21} + 6x_{22} + 6x_{23} + 9x_{24} \geq 32 = 4 \cdot 8$
		$8x_{21} + 6x_{22} + 6x_{23} + 9x_{24} \leq 48 = 6 \cdot 8$
Τύπου III	Βιταμίνες:	$8x_{31} + 6x_{32} + 10x_{33} + 4x_{34} \geq 36 = 4 \cdot 9$
		$8x_{31} + 6x_{32} + 1x_{33} + 4x_{34} \leq 54 = 6 \cdot 9$
	Πρωτεΐνες:	$10x_{31} + 5x_{32} + 12x_{33} + 8x_{34} \geq 54 = 6 \cdot 9$
	Ασβέστιο:	$6x_{31} + 10x_{32} + 6x_{33} + 6x_{34} \geq 54 = 6 \cdot 9$
	Λίπος:	$8x_{31} + 6x_{32} + 6x_{33} + 9x_{34} \geq 36 = 4 \cdot 9$
		$8x_{31} + 6x_{32} + 6x_{33} + 9x_{34} \leq 45 = 5 \cdot 9$

- Περιορισμοί προσήμου: Προφανώς η μάζα δεν μπορεί να έχει αρνητική τιμή: $x_{11}, x_{12}, x_{13}, x_{14}, x_{21}, x_{22}, x_{23}, x_{24}, x_{31}, x_{32}, x_{33}, x_{34} \geq 0$
- Αντικειμενική Συνάρτηση: Η τιμή ανά κιλό για τις πρώτες ύλες είναι **0.20, 0.12, 0.24, 0.12** αντίστοιχα. Εμείς θέλουμε την τιμή ανά τόνο οπότε παίρνουμε την αντικειμενική συνάρτηση $Z = 200(x_{11} + x_{21} + x_{31}) + 120(x_{12} + x_{22} + x_{32}) + 240(x_{13} + x_{23} + x_{33}) + (x_{14} + x_{24} + x_{34})$ και ισοδυναμεί με το κόστος παραγωγής.

Πρέπει λοιπόν τα $x_{11}, \dots, x_{34}$ να υπολογιστούν κατάλληλα, ούτως ώστε να ικανοποιούν όλους τους παραπάνω περιορισμούς, αλλά και να ελαχιστοποιούν την αντικειμενική συνάρτηση Z . Δηλαδή $x_{ij} = \arg \min_{x_{ij}} Z$, για όλα τα x_{ij} επιλεγμένα έτσι ώστε να ικανοποιούν όλους τους περιορισμούς.

4

Άσκηση 4

Από θεωρία γνωρίζουμε ότι ένα σύνολο $X \subset \mathbb{R}^n$ είναι κυρτό αν και μόνο αν:

$$\forall x, y \in X \text{ και } \forall \lambda \in [0, 1] : \lambda x + (1 - \lambda)y \in X \quad (4.1)$$

Η γεωμετρική ερμηνεία της παραπάνω εξίσωσης δηλώνει ότι όλα τα σημεία του ευθύγραμμου τμήματος που ορίζουν δύο οποιαδήποτε σημεία x, y του συνόλου X ανήκουν και αυτά στο σύνολο X .

4.1 (a)

$$A = \{(x_1, x_2) \in \mathbb{R}^2 \mid x_1^2 + x_2^2 \geq 3\}$$

Για τα παρακάτω σημεία x_0, y_0 στο σύνολο A , το σημείο P_0 στο ευθύγραμμο τμήμα x_0, y_0 για $\lambda = \frac{1}{2}$ δεν ανήκει μέσα στο σύνολο A :

$$x_0 = (1, -2), \quad 1^2 + (-2)^2 = 5 \geq 3 \Rightarrow x_0 \in A$$

$$y_0 = (-2, 1), \quad (-2)^2 + 1^2 = 5 \geq 3 \Rightarrow y_0 \in A$$

$$\lambda = \frac{1}{2} \in [0, 1]$$

$$P_0 = \lambda x_0 + (1 - \lambda)y_0$$

$$P_0 = \left(\frac{1}{2}, -1\right) + \left(-1, \frac{1}{2}\right)$$

$$P_0 = \left(-\frac{1}{2}, -\frac{1}{2}\right) \quad \left(-\frac{1}{2}\right)^2 + \left(-\frac{1}{2}\right)^2 = \frac{1}{2} < 3$$

Αφού $P_0 \notin A$ το σύνολο A δεν είναι κυρτό.

4.2 (b)

$B = \{(x_1, x_2, x_3) \in \mathbb{R}^3 | x_1 + 2x_2 \leq 1, \quad x_1 - 2x_3 \leq 2\}$ Θεωρώ όλα τα σημεία $P = (\rho_1, \rho_2, \rho_3)$ στο ευθύγραμμο τμήμα που ορίζουν 2 τυχαία σημεία $x = (x_1, x_2, x_3)$ και $y = (y_1, y_2, y_3)$ στο σύνολο B . Έστω $\lambda \in [0, 1]$

$$P = \lambda x + (1 - \lambda)y$$

$$\rho_1 = \lambda x_1 + (1 - \lambda)y_1$$

$$\rho_2 = \lambda x_2 + (1 - \lambda)y_2 \tag{4.2}$$

$$\rho_3 = \lambda x_3 + (1 - \lambda)y_3$$

Συνθήκη 1η:

$$\begin{array}{rcl} x_1 + 2x_2 \leq 1 & \xrightarrow{\lambda \geq 0} & \lambda x_1 + 2\lambda x_2 \leq \lambda \\ y_1 + 2y_2 \leq 1 & \xrightarrow{1-\lambda \geq 0} & (1 - \lambda)y_1 + 2(1 - \lambda)y_2 \leq 1 - \lambda \quad (+) \\ \hline \lambda x_1 + (1 - \lambda)y_1 + 2[\lambda x_2 + (1 - \lambda)y_2] & \leq & 1 \\ & \xrightarrow{(4.2)} & \rho_1 + 2\rho_2 \leq 1 \end{array}$$

Η 1η συνθήκη ικανοποιείται.

Συνθήκη 2η:

$$\begin{array}{rcl}
x_1 - 2x_3 \leq 2 & \xrightarrow{\lambda \geq 0} & \lambda x_1 - 2\lambda x_3 \leq 2\lambda \\
y_1 - 2y_3 \leq 2 & \xrightarrow{1-\lambda \geq 0} & (1-\lambda)y_1 - 2(1-\lambda)y_3 \leq 2 - 2\lambda \quad (+) \\
\hline
& & \lambda x_1 + (1-\lambda)y_1 - 2\lambda x_3 - 2(1-\lambda)y_3 \leq 2\lambda + 2 - 2\lambda \\
& & \xrightarrow{(4.2)} \rho_1 - 2\rho_3 \leq 2
\end{array}$$

Ικανοποιείται και η 2η συνθήκη. Συνεπώς όλα τα σημεία P ανήκουν στο σύνολο B , άρα το σύνολο B είναι κυρτό.

4.3 (c)

$C = \{(x_1, x_2, x_3) \in \mathbb{R}^3 | x_2 \geq x_1^2, \quad x_1 + 2x_2 + x_3 \leq 4\}$ Όμοια με παραπάνω παίρνουμε το σημείο P για το οποίο ισχύουν οι εξισώσεις (4.2)

Συνθήκη 1η:

$$\rho_1^2 = \lambda^2 x_1^2 + 2\lambda(1-\lambda)x_1 y_1 + (1-\lambda)^2 y_1^2 \quad (4.3)$$

Επίσης ισχύει:

$$\begin{array}{rcl}
x_2 \geq x_1^2 & \xrightarrow{\lambda \geq 0} & \lambda x_2 \geq \lambda x_1^2 \\
y_2 \geq y_1^2 & \xrightarrow{1-\lambda \geq 0} & (1-\lambda)y_2 \geq (1-\lambda)y_1^2 \quad (+) \\
\hline
& & \lambda x_2 + (1-\lambda)y_2 \geq \lambda x_1^2 + (1-\lambda)y_1^2 \\
& & \xrightarrow{(4.2)} \rho_2 \geq \lambda x_1^2 + (1-\lambda)y_1^2
\end{array}$$

Για να ικανοποιείται η 1η συνθήκη ($\rho_2 \geq \rho_1^2$) αρκεί Ν.Δ.Ο $\lambda x_1^2 + (1-\lambda)y_1^2 \geq \rho_1^2$ ή ισοδύναμα από (4.3):

$$\lambda x_1^2 + (1-\lambda)y_1^2 \geq \lambda^2 x_1^2 + 2\lambda(1-\lambda)x_1 y_1 + (1-\lambda)^2 y_1^2 \quad \Leftrightarrow$$

$$\lambda x_1^2 + (1-\lambda)y_1^2 - \lambda^2 x_1^2 - \lambda(1-\lambda)x_1 y_1 - \lambda(1-\lambda)x_1 y_1 - (1-\lambda)^2 y_1^2 \geq 0 \quad \Leftrightarrow$$

$$\lambda x_1[x_1 - \lambda x_1 - (1-\lambda)y_1] + (1-\lambda)y_1[y_1 - \lambda x_1 - (1-\lambda)y_1] \geq 0 \quad \Leftrightarrow$$

$$\lambda x_1(1-\lambda)(x_1 - y_1) - (1-\lambda)y_1\lambda(x_1 - y_1) \geq 0 \quad \Leftrightarrow$$

$$\lambda(1-\lambda)(x_1 - y_1)^2 \geq 0$$

Η παραπάνω έκφραση ισχύει αφού $\lambda, 1-\lambda, (x_1 - y_1)^2 \geq 0$, άρα η πρώτη συνθήκη ικανοποιείται.

Συνθήκη 2η:

$$x_1 + 2x_2 + x_3 \leq 4 \xrightarrow{\lambda \geq 0}$$

$$\lambda x_1 + 2\lambda x_2 + \lambda x_3 \leq 4\lambda$$

$$y_1 + 2y_2 + y_3 \leq 4 \xrightarrow{1-\lambda \geq 0} (1-\lambda)y_1 + 2(1-\lambda)y_2 + (1-\lambda)y_3 \leq 4(1-\lambda) \quad (+)$$

$$\lambda x_1 + (1-\lambda)y_1 + 2[\lambda x_2 + (1-\lambda)y_2] + \lambda x_3 + (1-\lambda)y_3 \leq 4$$

$$\xrightarrow{(4.2)} \rho_1 + 2\rho_2 + \rho_3 \leq 4$$

Ικανοποιείται και η 2η συνθήκη, συνεπώς το σύνολο C είναι κυρτό.

4.4 (d)

$D = \{(x_1, x_2, x_3) \in \mathbb{R}^3 | x_3 = |x_2|, \quad x_1 \leq 3\}$ Έστω $x_0 = (x_1, x_2, x_3) = (2, 2, 2)$
και $y_0 = (y_1, y_2, y_3) = (2, -2, 2)$. Ισχύουν:

$$\left. \begin{array}{l} x_3 = 2 = |2| = |x_2| \\ x_1 = 2 \leq 3 \end{array} \right\} \Rightarrow x_0 \in D$$

$$\left. \begin{array}{l} y_3 = 2 = |-2| = |y_2| \\ y_1 = 2 \leq 3 \end{array} \right\} \Rightarrow y_0 \in D$$

Παίρνω $P_0 = (\rho_1, \rho_2, \rho_3) = \lambda x_0 + (1 - \lambda)y_0$ για $\lambda = \frac{1}{2}$, $\lambda \in [0, 1]$. Δηλαδή, το P_0 βρίσκεται στο ευθύγραμμο τμήμα που ορίζουν τα x_0, y_0 .

$$\left. \begin{array}{l} \rho_2 = \lambda x_2 + (1 - \lambda)y_2 = \frac{1}{2}2 + (1 - \frac{1}{2})(-2) = 1 - 1 = 0 \\ \rho_3 = \lambda x_3 + (1 - \lambda)y_3 = \frac{1}{2}2 + (1 - \frac{1}{2})(2) = 1 + 1 = 2 \end{array} \right\} \Rightarrow \rho_3 \neq |\rho_2| \Rightarrow P_0 \notin D$$

Συνεπώς η (4.1) δεν ικανοποιείται και το σύνολο D είναι μη κυρτό.

5

Άσκηση 5

5.1 Θεωρητικό Υπόβαθρο

Από θεωρία γνωρίζουμε ότι ένα **Υπερεπίπεδο στον χώρο** $\mathbb{R}^n$ ορίζεται:

$$\begin{aligned} H &= \{x \in \mathbb{R}^n | a^T x = b\}, \quad b \in \mathbb{R}, a = [a_1 \dots a_n]^T \in \mathbb{R}^n, a \neq 0 \\ H^+ &= \{x \in \mathbb{R}^n | a^T x \leq b\} \quad \text{Πρώτος ημιχώρος που ορίζει το } H \\ H^- &= \{x \in \mathbb{R}^n | a^T x \geq b\} \quad \text{Δεύτερος ημιχώρος που ορίζει το } H \end{aligned} \quad (5.1)$$

Το διάνυσμα a είναι ορθογώνιο στο H και ονομάζεται κανονικό.

Ένα **Πολύτοπο στο** $\mathbb{R}^n$ ορίζεται:

$$\{x \in \mathbb{R}^n : Ax \leq b\} \quad (5.2)$$

Η **Εφικτή περιοχή** F στο $\mathbb{R}^n$ ορίζεται:

$$F = \{x \in \mathbb{R}^n : Ax \leq b, x \geq 0\} \quad (5.3)$$

Ένα σημείο x^0 στην εφικτή περιοχή

$$F = \bigcap_{i=1}^P H_i^+ \neq \emptyset$$

που ορίζεται από p ημιχώρους $H_1^+, \dots, H_p^+ \in \mathbb{R}^n$ είναι **κορυφή της εφικτής περιοχής** αν και μόνο αν το x^0 είναι σημείο τομής n ανεξάρτητων υπερεπιπέδων $H_i, i \in \{1, \dots, p\}$ που αντιστοιχούν σε ημιχώρους H^+ . Αν μια κορυφή είναι το σημείο τομής περισσότερων από n ανεξάρτητων υπερεπιπέδων, τότε είναι **εκφυλισμένη**

Επιπλέον γνωρίζουμε ότι κάθε βασική εφικτή λύση του προβλήματος αντιστοιχεί σε μία (ακριβώς) κορυφή της F και αντιστρόφως κάθε κορυφή αντιστοιχεί σε τουλάχιστον μία βασική εφικτή λύση.

Τέλος, όσον αφορά την γραμμική ανεξαρτησία, γνωρίζουμε ότι ένα σύνολο διανυσμάτων $a_1, a_2, \dots, a_r \in \mathbb{R}^n$ είναι γραμμικά εξαρτημένα αν και μόνο αν υπάρχουν αριθμοί $\lambda_1, \lambda_2, \dots, \lambda_r$ **όχι όλοι μηδέν** έτσι ώστε:

$$\sum_{k=1}^r \lambda_k a_k = 0$$

Αλλιώς τα $a_1, a_2, \dots, a_r$ είναι γραμμικά ανεξάρτητα. Το παραπάνω άθροισμα θα μπορούσε να θεωρηθεί ένα ομογενές σύστημα $Ax = 0$ με $x = [\lambda_1, \lambda_2, \dots, \lambda_r]^T$ και $A = [a_1, a_2, \dots, a_r]$ (θεωρώντας ότι τα διανύσματα είναι κάθετα). Για να έχει μοναδική λύση την μηδενική, η ορίζουσα του πίνακα A πρέπει να είναι διάφορη του 0.

Την παραπάνω ιδιότητα την εκμεταλλευόμαστε για να εξετάσουμε την ανεξαρτησία των υπερεπιπέδων, για την εύρεση των κορυφών. k υπερεπίπεδα $H_1, \dots, H_k$ ονομάζονται ανεξάρτητα αν και μόνο αν τα αντίστοιχα κανονικά τους διανύσματα είναι γραμμικά ανεξάρτητα.

5.2 α

Ορίζουμε λοιπόν τα υπερεπίπεδα από τις ανισώσεις του προβλήματος:

$$H_1 : \quad x_1 + x_2 = 10$$

$$H_2 : \quad x_2 + x_3 = 15$$

$$H_3 : \quad x_1 + x_3 = 12$$

$$H_4 : \quad 20x_1 + 10x_2 + 15x_3 = 300$$

$$H_5 : \quad x_1 = 0$$

$$H_6 : \quad x_2 = 0$$

$$H_7 : \quad x_3 = 0$$

Θα βρούμε τις τομές ανά 3 υπερεπιπέδων καθώς είμαστε στο $\mathbb{R}^3$, λόγω των τριών μεταβλητών x_1, x_2, x_3 , όταν τα υπερεπίπεδα είναι ανεξάρτητα μεταξύ τους.

5.2.1 Κώδικας

```
# Έλεγχος περιορισμών
def check_constraints(x, geq, leq, eq, sign):
    x = np.array(x)
    for const in geq:
        if np.dot(np.array(const[:-1]), x) < const[-1]:
            #print(const)
            return False
    for const in leq:
        if np.dot(np.array(const[:-1]), x) > const[-1]:
            #print(const)
            return False
    for const in eq:
        if np.dot(np.array(const[:-1]), x) != const[-1]:
            #print(const)
            return False
```

```
for const in sign:
    if np.dot(np.array(const[:-1]),x)<0:
        #print(const)
        return False
    return True

#Εκφυλισμένες Κορυφές

def degenerated_vertexes(*vertexes):
    dv=[]
    for i in range(len(vertexes)):
        for j in range(i+1, len(vertexes)):
            if vertexes[i]==vertexes[j] and vertexes[i] not in dv:
                dv.append(vertexes[i])
    return dv

constraints_geq5 = [(1,1,0,10),
                    (0,1,1,15),
                    (1,0,1,12)]
constraints_leq5 = [(20,10,15,300)]
constraints_eq5 = []
constraints_sign5 = [(1,0,0,0),
                     (0,1,0,0),
                     (0,0,1,0)]
hyperplanes = (constraints_geq5+
               constraints_leq5+
               constraints_eq5+
               constraints_sign5)
vertexes5 = []
counter = 0

print('Γραμμικά Εξαρτημένα Τετραπύεδα:')
for i in range(len(hyperplanes)):
    for j in range(i+1,len(hyperplanes)):
        for k in range(j+1,len(hyperplanes)):
            h1 = hyperplanes[i]
            h2 = hyperplanes[j]
            h3 = hyperplanes[k]
            v = vertex(h1,h2,h3)
```

```

    if v!=-1:    # Μη μηδενική ορίζουσα
        vertexes5.append(v)
    else:       # Ποιες τριάδες υπερεπιπέδων είναι εξαρτημένες
        print(h1,h2,h3)
print()
c=1
for v in vertexes5:
    v = tuple(float('{:.2f}'.format(float(v_f)))) for v_f in v)
    print('Κορυφή {:2d}:'.format(c),v)
    c+=1
print()
c=1

print('Κορυφές στην εφικτή περιοχή')

#Κορυφές στην εφικτή περιοχή
feasible_v = []

for v in vertexes5:
    if check_constraints(v,constraints_geq5, constraints_leq5, constraints_eq5, constraints_sign5):
        v = tuple(float('{:.2f}'.format(float(v_f)))) for v_f in v)
        feasible_v.append(v)
        print('Κορυφή {:2d}:'.format(c),v)
        c+=1
print()
print('Εκφυλισμένες κορυφές')
print(degenerated_vertexes(*vertexes5))
print('Εκφυλισμένες κορυφές στην εφικτή περιοχή')
print(degenerated_vertexes(*feasible_v))

```

Η συνάρτηση **vertex** έχει οριστεί στην άσκηση 1

Η συνάρτηση **degenerated_vertexes** παίρνει σαν ορίσματα διάφορες κορυφές. Αν κάποιες από αυτές τις κορυφές είναι ίδιες τότε τις προσθέτει σε μία λίστα (την **dv**) και επιστρέφει αυτήν την λίστα.

5.2.2 Αποτελέσματα

Έξοδος:

Γραμμικά Εξαρτημένα Υπερεπίπεδα:

(1, 1, 0, 10) (1, 0, 0, 0) (0, 1, 0, 0)
(0, 1, 1, 15) (0, 1, 0, 0) (0, 0, 1, 0)
(1, 0, 1, 12) (1, 0, 0, 0) (0, 0, 1, 0)

Κορυφή 1: (3.5, 6.5, 8.5)
Κορυφή 2: (5.0, 5.0, 10.0)
Κορυφή 3: (0.0, 10.0, 5.0)
Κορυφή 4: (10.0, 0.0, 15.0)
Κορυφή 5: (-5.0, 15.0, 0.0)
Κορυφή 6: (-4.0, 14.0, 16.0)
Κορυφή 7: (0.0, 10.0, 12.0)
Κορυφή 8: (10.0, -0.0, 2.0)
Κορυφή 9: (12.0, -2.0, 0.0)
Κορυφή 10: (0.0, 10.0, 13.33)
Κορυφή 11: (10.0, 0.0, 6.67)
Κορυφή 12: (20.0, -10.0, 0.0)
Κορυφή 13: (0.0, 10.0, 0.0)
Κορυφή 14: (10.0, 0.0, 0.0)
Κορυφή 15: (6.0, 9.0, 6.0)
Κορυφή 16: (0.0, 3.0, 12.0)
Κορυφή 17: (-3.0, 0.0, 15.0)
Κορυφή 18: (12.0, 15.0, 0.0)
Κορυφή 19: (0.0, -15.0, 30.0)
Κορυφή 20: (3.75, 0.0, 15.0)
Κορυφή 21: (7.5, 15.0, 0.0)
Κορυφή 22: (0.0, 0.0, 15.0)
Κορυφή 23: (0.0, 15.0, 0.0)
Κορυφή 24: (0.0, 12.0, 12.0)
Κορυφή 25: (24.0, 0.0, -12.0)
Κορυφή 26: (12.0, 6.0, 0.0)
Κορυφή 27: (0.0, 0.0, 12.0)

Κορυφή 28: (12.0, 0.0, 0.0)
 Κορυφή 29: (0.0, 0.0, 20.0)
 Κορυφή 30: (0.0, 30.0, 0.0)
 Κορυφή 31: (15.0, 0.0, 0.0)
 Κορυφή 32: (0.0, 0.0, 0.0)

Κορυφές στην εφικτή περιοχή
 Κορυφή 1: (3.5, 6.5, 8.5)
 Κορυφή 2: (5.0, 5.0, 10.0)
 Κορυφή 7: (0.0, 10.0, 12.0)
 Κορυφή 10: (0.0, 10.0, 13.33)
 Κορυφή 15: (6.0, 9.0, 6.0)
 Κορυφή 24: (0.0, 12.0, 12.0)

Εκφυλισμένες κορυφές

[]

Εκφυλισμένες κορυφές στην εφικτή περιοχή

[]

Βλέπουμε ότι είναι 32 οι κορυφές και άλλοι 3 συνδιασμοί των υπερεπιπέδων, που όμως πρόκειται για εξαρτημένα υπερεπίπεδα. Σύνολο 35 συνδιασμοί υπερεπιπέδων που προκύπτουν από 7 περιορισμούς. Αφού επιλέγουμε 3 από τα 7 υπερεπίπεδα κάθε φορά, οι συνδιασμοί είναι $\binom{7}{3} = \frac{7!}{3!4!} = 35$. Τέλος, καμία κορυφή δεν είναι εκφυλισμένη.

5.3 β

Προσθέτουμε τις μεταβλητές χαλάρωσης στο σύστημα ανισώσεων, για να μετατρέψουμε τις ανισώσεις σε εξισώσεις. Οι μεταβλητές χαλάρωσης, θα είναι μη αρνητικές και συνεπώς όταν έχουμε ανισωτικές σχέσεις του τύπου $Ax \leq b$ θα προστίθενται με θετικό πρόσημο, ενώ για ανισωτικές σχέσεις του τύπου $Ax \geq b$ θα προστίθενται με αρνητικό πρόσημο. Το

σύστημα ανισώσεων που δίνεται τροποποιείται ως εξής:

$$\begin{array}{cccccccl}
 x_1 & & +x_2 & & -x_4 & & & = 10 \\
 & & & x_2 & +x_3 & & -x_5 & = 15 \\
 x_1 & & & +x_3 & & & -x_6 & = 12 \\
 20x_1 & +10x_2 & +15x_3 & & & & & +x_7 = 300
 \end{array}$$

Αυτό είναι ένα σύστημα $Ax = b$. Επιλύουμε το παραπάνω σύστημα για διαφορετικούς συνδιασμούς των στηλών του πίνακα A ορίζοντας τον βασικό πίνακα $B_{4 \times 4}$ και λύνοντας το σύστημα $Bx = b \Leftrightarrow x = B^{-1}b$, εφόσον ο B είναι αντιστρέψιμος δηλαδή $\det(B) \neq 0$. Οι υπόλοιπες στήλες θα αντιστοιχούν στις μη βασικές μεταβλητές που θα πάρουν την τιμή 0. Τέλος στην επίλυση του συστήματος, θα πρέπει οι λύσεις να είναι μη αρνητικές, για να ικανοποιούν τους περιορισμούς προσήμου.

5.3.1 Κώδικας

Υπερεπίπεδα από περιορισμούς προσήμου

```
slack_vars = [(-1,0,0,0),(0,-1,0,0),(0,0,-1,0),(0,0,0,1)]
```

ορισμός πινάκων για το σύστημα $Ax=b$

A προκύπτει από περιορισμούς

```
A1 = np.array(constraints_geq5)[:,-1]
```

```
A2 = np.array(constraints_leq5)[:,-1]
```

```
#A3 = np.array(constraints_eq5)[:,-1]
```

```
A = np.vstack((A1,A2))
```

```
b1 = np.array((constraints_geq5))[:,-1]
```

```
b2 = np.array((constraints_leq5))[:,-1]
```

```
#b3 = np.array((constraints_eq5))[:,-1]
```

```
b = np.hstack((b1, b2)).T
```

```
slack_vars_matrice = np.array(slack_vars)
```

```
A_last = np.hstack((A,slack_vars_matrice))
```

```
columns = A_last.shape[1]
v={}
vertexes = []
degenerated_vertexes5 = []
basic_vals = []

for i in range(columns):
    for j in range(i+1, columns):
        for k in range(j+1, columns):
            for l in range(k+1, columns):

                # ορισμός βασικού πίνακα για διαφορετικές στήλες του A
                B = np.array((A_last[:, [i,j,k,l]]))

                # ορισμός βασικών μεταβλητών
                basic_vars = i+1,j+1,k+1,l+1
                # ορισμός μη βασικών μεταβλητών
                non_basic_vars = tuple([index+1 for index in range(columns) if index+1 not in basic_vars])

                # Επίλυση του συστήματος Ax=b
                if np.linalg.det(B)!=0:

                    # Αντίστροφος B
                    B_inv = np.linalg.inv(B)

                    # Τιμές βασικών μεταβλητών
                    basic_values = B_inv@b

                    # οι εριορισμοί προσήμου θα καθορίσουν την εφικτότητα της λύσης
                    if np.all(basic_values>=0):
                        accepted = 'Βασική εφικτή λύση'

                    # Αν υπάρχει έστω και ένα 0 η λύση μου είναι εκφυλισμένη
                    if not np.all(basic_values>0):
                        degenerated_vertexes5.append(basic_vars)
                    else:
                        basic_vals.append(tuple(basic_values))
```

```

else:
    accepted = 'Βασική μη εφικτή λύση'

    basic_values = tuple(basic_values)
    print("Βασικές μεταβλητές:")
    for index in range(len(basic_vars)):
        print('x_{0} = {1:.2f}'.format(basic_vars[index], basic_values[index]), end = '')
        v[basic_vars[index]]=basic_values[index]
    print()
    print(accepted)
    if accepted=='Βασική εφικτή λύση':
        print("Μη βασικές μεταβλητές:")
        for index in range(len(non_basic_vars)):
            print('x_{0} = 0'.format(non_basic_vars[index]), end = '')
            v[non_basic_vars[index]]=0
        print()
        vertexes.append([v[index] for index in range(1,4)])
    print()
else:
    print('Αδύνατο σύστημα για βασικές μεταβλητές: ', end = '')
    for var in basic_vars:
        print('x_{0}'.format(var), end='')
    print('\n')

```

5.3.2 Αποτελέσματα

Βασικές μεταβλητές:

$x_1 = 6.00$ $x_2 = 9.00$ $x_3 = 6.00$ $x_4 = 5.00$

Βασική εφικτή λύση

Μη βασικές μεταβλητές:

$x_5 = 0$ $x_6 = 0$ $x_7 = 0$

Βασικές μεταβλητές:

$x_1 = -4.00$ $x_2 = 14.00$ $x_3 = 16.00$ $x_5 = 15.00$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 5.00 \quad x_2 = 5.00 \quad x_3 = 10.00 \quad x_6 = 3.00$$

Βασική εφικτή λύση

Μη βασικές μεταβλητές:

$$x_4 = 0 \quad x_5 = 0 \quad x_7 = 0$$

Βασικές μεταβλητές:

$$x_1 = 3.50 \quad x_2 = 6.50 \quad x_3 = 8.50 \quad x_7 = 37.50$$

Βασική εφικτή λύση

Μη βασικές μεταβλητές:

$$x_4 = 0 \quad x_5 = 0 \quad x_6 = 0$$

Βασικές μεταβλητές:

$$x_1 = 12.00 \quad x_2 = 6.00 \quad x_4 = 8.00 \quad x_5 = -9.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 7.50 \quad x_2 = 15.00 \quad x_4 = 12.50 \quad x_6 = -4.50$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 12.00 \quad x_2 = 15.00 \quad x_4 = 17.00 \quad x_7 = -90.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 20.00 \quad x_2 = -10.00 \quad x_5 = -25.00 \quad x_6 = 8.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 12.00 \quad x_2 = -2.00 \quad x_5 = -17.00 \quad x_7 = 80.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = -5.00 \quad x_2 = 15.00 \quad x_6 = -17.00 \quad x_7 = 250.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 24.00 \quad x_3 = -12.00 \quad x_4 = 14.00 \quad x_5 = -27.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 3.75 \quad x_3 = 15.00 \quad x_4 = -6.25 \quad x_6 = 6.75$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = -3.00 \quad x_3 = 15.00 \quad x_4 = -13.00 \quad x_7 = 135.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 10.00 \quad x_3 = 6.67 \quad x_5 = -8.33 \quad x_6 = 4.67$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 10.00 \quad x_3 = 2.00 \quad x_5 = -13.00 \quad x_7 = 70.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 10.00 \quad x_3 = 15.00 \quad x_6 = 13.00 \quad x_7 = -125.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 15.00 \quad x_4 = 5.00 \quad x_5 = -15.00 \quad x_6 = 3.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_1 = 12.00 \quad x_4 = 2.00 \quad x_5 = -15.00 \quad x_7 = 60.00$$

Βασική μη εφικτή λύση

Αδύνατο σύστημα για βασικές μεταβλητές: $x_1 \ x_4 \ x_6 \ x_7$

Βασικές μεταβλητές:

$$x_1 = 10.00 \quad x_5 = -15.00 \quad x_6 = -2.00 \quad x_7 = 100.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_2 = 12.00 \quad x_3 = 12.00 \quad x_4 = 2.00 \quad x_5 = 9.00$$

Βασική εφικτή λύση

Μη βασικές μεταβλητές:

$$x_1 = 0 \quad x_6 = 0 \quad x_7 = 0$$

Βασικές μεταβλητές:

$$x_2 = -15.00 \quad x_3 = 30.00 \quad x_4 = -25.00 \quad x_6 = 18.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_2 = 3.00 \quad x_3 = 12.00 \quad x_4 = -7.00 \quad x_7 = 90.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_2 = 10.00 \quad x_3 = 13.33 \quad x_5 = 8.33 \quad x_6 = 1.33$$

Βασική εφικτή λύση

Μη βασικές μεταβλητές:

$$x_1 = 0 \quad x_4 = 0 \quad x_7 = 0$$

Βασικές μεταβλητές:

$$x_2 = 10.00 \quad x_3 = 12.00 \quad x_5 = 7.00 \quad x_7 = 20.00$$

Βασική εφικτή λύση

Μη βασικές μεταβλητές:

$$x_1 = 0 \quad x_4 = 0 \quad x_6 = 0$$

Βασικές μεταβλητές:

$$x_2 = 10.00 \quad x_3 = 5.00 \quad x_6 = -7.00 \quad x_7 = 125.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_2 = 30.00 \quad x_4 = 20.00 \quad x_5 = 15.00 \quad x_6 = -12.00$$

Βασική μη εφικτή λύση

Αδύνατο σύστημα για βασικές μεταβλητές: $x_2 \ x_4 \ x_5 \ x_7$

Βασικές μεταβλητές:

$$x_2 = 15.00 \quad x_4 = 5.00 \quad x_6 = -12.00 \quad x_7 = 150.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_2 = 10.00 \quad x_5 = -5.00 \quad x_6 = -12.00 \quad x_7 = 200.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_3 = 20.00 \quad x_4 = -10.00 \quad x_5 = 5.00 \quad x_6 = 8.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$$x_3 = 12.00 \quad x_4 = -10.00 \quad x_5 = -3.00 \quad x_7 = 120.00$$

Βασική μη εφικτή λύση

Βασικές μεταβλητές:

$x_3 = 15.00$ $x_4 = -10.00$ $x_6 = 3.00$ $x_7 = 75.00$

Βασική μη εφικτή λύση

Αδύνατο σύστημα για βασικές μεταβλητές: x_3 x_5 x_6 x_7

Βασικές μεταβλητές:

$x_4 = -10.00$ $x_5 = -15.00$ $x_6 = -12.00$ $x_7 = 300.00$

Βασική μη εφικτή λύση

5.4 γ

Στον κώδικα του ερωτήματος β 5.3.1, ορίστηκε μια λίστα ερτεξ, στην οποία προσθέταμε τις τιμές x_1, x_2, x_3 , στην περίπτωση στην οποία είχαμε εφικτή λύση. Τώρα θα τυπώσουμε τα αποτελέσματα του ερωτήματος 5.3 β, που αντιστοιχούν και αυτά σε κορυφές x_1, x_2, x_3 και έπειτα τις κορυφές στην εφικτή περιοχή που βρήκαμε στο ερώτημα 5.2 α. Παρατηρούμε ότι είναι οι ίδιες κορυφές, που τυπώνονται με διαφορετική σειρά. Έπειτα ορίζουμε την αντικειμενική συνάρτηση Z και βρίσκουμε τις κορυφές που την ελαχιστοποιούν.

5.4.1 Κώδικας

```
c = 0
for v in vertexes:
    c+=1
    v = tuple(float('{:.2f}'.format(float(v_f))) for v_f in v)
    print('Κορυφή {}:'.format(c),v)
c = 0
print()
for b in basic_vals:
    c+=1
    b = tuple(float('{:.2f}'.format(float(b_f))) for b_f in b)
    print('Βασική λύση που αντιστοιχεί στην κορυφή {}:'.format(c),b)
c=1
```

```

print('Εκφυλισμένες κορυφές: ', degenerated_vertexes5)
print()
for v in vertexes5:
    if check_constraints(v, constraints_geq5, constraints_leq5, constraints_eq5, constraints_sign5):
        v = tuple(float('{:.2f}'.format(float(v_f))) for v_f in v)
        feasible_v.append(v)
        print('Κορυφή {:2d}:".format(c), v)
    c+=1
print('Εκφυλισμένες κορυφές: ', degenerated_vertexes(*vertexes5))

def Z5(x1,x2,x3):
    return 8*x1+5*x2+4*x3
min_Z5 = np.inf
best_vertexes = []
for v in vertexes:
    v = tuple(float('{:.2f}'.format(float(v_f))) for v_f in v)
    if Z5(*v)<=min_Z5:
        min_Z5 = Z5(*v)
        best_vertexes.clear()
        best_vertexes.append(v)
print()
print(min_Z5)
print('Βέλτιστη/ες κορυφή/ές:', *best_vertexes, sep='\n')

```

5.4.2 Αποτελέσματα

Κορυφή 1: (6.0, 9.0, 6.0)
 Κορυφή 2: (5.0, 5.0, 10.0)
 Κορυφή 3: (3.5, 6.5, 8.5)
 Κορυφή 4: (0.0, 12.0, 12.0)
 Κορυφή 5: (0.0, 10.0, 13.33)
 Κορυφή 6: (0.0, 10.0, 12.0)

Βασική λύση που αντιστοιχεί στην κορυφή 1: (6.0, 9.0, 6.0, 5.0)
 Βασική λύση που αντιστοιχεί στην κορυφή 2: (5.0, 5.0, 10.0, 3.0)
 Βασική λύση που αντιστοιχεί στην κορυφή 3: (3.5, 6.5, 8.5, 37.5)

Βασική λύση που αντιστοιχεί στην κορυφή 4: (12.0, 12.0, 2.0, 9.0)

Βασική λύση που αντιστοιχεί στην κορυφή 5: (10.0, 13.33, 8.33, 1.33)

Βασική λύση που αντιστοιχεί στην κορυφή 6: (10.0, 12.0, 7.0, 20.0)

Εκφυλισμένες κορυφές: []

Κορυφή 1: (3.5, 6.5, 8.5)

Κορυφή 2: (5.0, 5.0, 10.0)

Κορυφή 7: (0.0, 10.0, 12.0)

Κορυφή 10: (0.0, 10.0, 13.33)

Κορυφή 15: (6.0, 9.0, 6.0)

Κορυφή 24: (0.0, 12.0, 12.0)

Εκφυλισμένες κορυφές: []

94.5

Βέλτιστη/ες κορυφή/ές:

(3.5, 6.5, 8.5)

6

Άσκηση 6

6.1 α

Προσθέτουμε τις μεταβλητές χαλάρωσης στο πρόβλημα που δίνεται και καλούμαστε να επιλύσουμε το παρακάτω πρόβλημα:

$$\max Z = 2x_1 + x_2 + 6x_3 - x_4$$

όταν

$$x_1 + 2x_2 + 4x_3 - x_4 + x_5 = 6$$

$$2x_1 + 3x_2 - x_3 + x_4 + x_6 = 12$$

$$x_1 + x_3 + x_4 + x_7 = 2$$

$$x_1, x_2, x_3, x_4, x_5, x_6, x_7 \geq 0$$

Πλέον έχουμε να λύσουμε ένα σύστημα της μορφής $Ax = b$ με $Z = cx$. Για τον αλγόριθμο simplex, φτιάχνουμε μια λίστα με τις βασικές μεταβλητές, που αρχικά είναι οι μεταβλητές χαλάρωσης. Ορίζουμε έναν πίνακα που θα τον ονομάσουμε **simplex_matrix** του οποίου οι στήλες θα καθορίζονται από τις μεταβλητές x και μία στήλη για το b και οι γραμμές από το πλήθος των βασικών μεταβλητών και άλλη μία για την αντικειμενική συνάρτηση Z , που παίρνουμε την αρνητική της τιμή. Οι στήλες των βασικών μεταβλητών θα πρέπει να σχηματίζουν τον μοναδιαίο πίνακα. Έτσι αρχικοποιούμε τις βασικές μεταβλητές να είναι οι μεταβλητές χαλάρωσης. Το νέο στοιχείο που θα μετατραπεί σε βασική μεταβλητή είναι

ένα οποιοδήποτε στοιχείο, με θετικό συντελεστή c για την αντικειμενική συνάρτηση. Εφαρμόζουμε το κριτήριο ελαχίστου λόγου και βρίσκουμε ποιο στοιχείο θα βγει από τις βασικές μεταβλητές. Η στήλη του στοιχείου που θα μπει και η γραμμή του στοιχείου που θα βγει καθορίζουν το οδηγό στοιχείο και βάσει αυτού θα αλλάξουν τα υπόλοιπα στοιχεία του `simplex_matrix`. Στο ερώτημα αυτό, η επιλογή του εισερχόμενου στοιχείου έγινε με βάση τον μεγαλύτερο συντελεστή του Z .

6.1.1 Κώδικας

```
def simplex_step(simplex_Matrix, basic_vars, choose_ins_var = 'max_c'):
    BN = simplex_Matrix[1:, :-1]
    b = simplex_Matrix[1:, -1:]
    BNb = simplex_Matrix[1:, :]
    c = simplex_Matrix[0, :-1]
    Z = simplex_Matrix[0, -1]
    if np.all(c <= 0):
        return -1,-1,simplex_Matrix, basic_vars
    ins_ind = np.argmax(c)
    if np.all(BN[:, ins_ind] <= 0):
        return -2,-2,simplex_Matrix, basic_vars
    if choose_ins_var == 'max_c':
        ins_ind = np.argmax(c)
    else:
        ins_ind = choose_ins_var
    min_ratio = np.inf
    for i in range(BN.shape[0]):
        if BN[i][ins_ind]>0:
            ratio = b[i]/BN[i][ins_ind]
            if ratio<min_ratio:
                min_ratio = ratio
                out_ind = i
    # ratios = np.divide(b.flatten(), BN[:, ins_ind], where=(BN[:, ins_ind] > 0))
    # print(ratios)
    # out_ind = np.argmin(ratios)
    out_element = basic_vars[out_ind]
    basic_vars[out_ind]=ins_ind
    pivot = simplex_Matrix[out_ind+1][ins_ind]
```

```

    #rows different from pivot
    for i in range(simplex_Matrix.shape[0]):
        if i != out_ind + 1:
            simplex_Matrix[i, :] -= simplex_Matrix[i, ins_ind] * simplex_Matrix[out_ind + 1, :]/pivot

    #rows same as pivot
    simplex_Matrix[out_ind + 1, :] /= pivot
    return ins_ind, out_element, simplex_Matrix, basic_vars

vars_indexes = np.linspace(0, 6, 7, dtype='int')
basic_vars_indexes = np.linspace(4, 6, 3, dtype='int')

BN = np.array([[1, 2, 4, -1, 1, 0, 0],
               [2, 3, -1, 1, 0, 1, 0],
               [1, 0, 1, 1, 0, 0, 1]])
b = np.array([[6], [12], [2]])
c = np.array([[2, 1, 6, -4, 0, 0, 0]])
Z = np.array([[0]])
cZ = np.hstack((c, Z))
BNb = np.hstack((BN, b))
simplex_Matrix = np.vstack((cZ, BNb))
simplex_Matrix = np.array(simplex_Matrix, dtype='float64')

max_steps = 15
step = 0
print('Βήμα:', step)
while step <= max_steps:
    print('Βασικές μεταβλητές: x{}, x{}, x{}'.format(*basic_vars_indexes+1))
    print(np.round(simplex_Matrix, decimals=3))
    ins_ind, out_element, simplex_Matrix, basic_vars = simplex_step(simplex_Matrix, basic_vars_indexes)
    if ins_ind == -1:
        print('Βρέθηκε η βέλτιστη λύση')
        break
    elif ins_ind == -2:
        print('Το πρόβλημα είναι μη φραγμένο.')
        break
    step += 1
print()

```

```

print('Βήμα:', step)
print('Εισερχόμενη μεταβλητή: x{}'.format(ins_ind + 1))
print('Εξερχόμενη μεταβλητή: x{}'.format(out_element+1))

print('\nΤελική Λύση:')
for i in range(BN.shape[1]):
    if i in basic_vars_indexes:
        idx = np.where(basic_vars_indexes == i)[0][0]
        print('x{} ='.format(i+1), np.round(simplex_Matrix[idx + 1, -1], decimals=2))
    else:
        print('x{} = 0'.format(i+1))
print('Z =', -np.round(simplex_Matrix[0, -1], decimals=2)) # Προσοχή στο πρόσημο του Z

```

6.1.2 Αποτελέσματα

Βήμα: 0

Βασικές μεταβλητές: x5,x6,x7

```

[[ 2.  1.  6. -4.  0.  0.  0.  0.]
 [ 1.  2.  4. -1.  1.  0.  0.  6.]
 [ 2.  3. -1.  1.  0.  1.  0. 12.]
 [ 1.  0.  1.  1.  0.  0.  1.  2.]]

```

Βήμα: 1

Εισερχόμενη μεταβλητή: x3

Εξερχόμενη μεταβλητή: x5

Βασικές μεταβλητές: x3,x6,x7

```

[[ 0.5 -2.  0. -2.5 -1.5  0.  0. -9. ]
 [ 0.25 0.5  1. -0.25 0.25  0.  0.  1.5 ]
 [ 2.25 3.5  0.  0.75 0.25  1.  0. 13.5 ]
 [ 0.75 -0.5  0.  1.25 -0.25  0.  1.  0.5 ]]

```

Βήμα: 2

Εισερχόμενη μεταβλητή: x1

Εξερχόμενη μεταβλητή: x7

Βασικές μεταβλητές: x_3, x_6, x_1

```
[[ 0.    -1.667  0.    -3.333 -1.333  0.    -0.667 -9.333]
 [ 0.     0.667  1.    -0.667  0.333  0.    -0.333  1.333]
 [ 0.     5.     0.    -3.     1.     1.    -3.     12.   ]
 [ 1.    -0.667  0.     1.667 -0.333  0.     1.333  0.667]]
```

Βρέθηκε η βέλτιστη λύση

Τελική Λύση:

$x_1 = 0.67$

$x_2 = 0$

$x_3 = 1.33$

$x_4 = 0$

$x_5 = 0$

$x_6 = 12.0$

$x_7 = 0$

$Z = 9.33$

6.2 β

Έγινε μια προσπάθεια καταγραφής του ιστορικού της εφαρμογής σιμπλεξ, για διάφορες επιλογές εισερχόμενων στοιχείων. Όμως δυστυχώς η λύση δεν έφτασε στο τέλος της. Παρακάτω παρατίθεται αυτή η προσπάθεια:

```
def simplex_with_history(simplex_Matrix, basic_vars, history=None, previous_state=None):
    if history is None:
        history = []

    current_state = {
        'matrix': simplex_Matrix.copy(),
        'basic_vars': basic_vars.copy(),
        'possible_next_states': [],
        'optimal': False,
        'previous': previous_state # Πίναδας στην προηγούμενη κατάσταση
    }
```

```

c = simplex_Matrix[0, :-1]

# Έλεγχος για βέλτιστη λύση
if np.all(c <= 1e-8):
    current_state['optimal'] = True
    # Αναδρομική ενημέρωση προηγούμενων καταστάσεων
    update_previous_states(current_state)
    history.append(current_state)
    return (True, -1, -1, simplex_Matrix, basic_vars, history)

possible_ins_ind = np.where(c > 1e-8)[0]

for ins_ind in possible_ins_ind:
    temp_matrix = simplex_Matrix.copy()
    temp_basic = basic_vars.copy()
    entering, leaving, new_matrix, new_basic = simplex_step(temp_matrix, temp_basic, ins_ind)

    next_state = {
        'matrix': new_matrix.copy(),
        'basic_vars': new_basic.copy(),
        'ins_var': entering,
        'out_var': leaving,
        'optimal': False,
        'previous': current_state # Δείκτης στην τρέχουσα κατάσταση
    }
    current_state['possible_next_states'].append(next_state)

history.append(current_state)

if not current_state['possible_next_states']:
    return (False, -2, -2, simplex_Matrix, basic_vars, history)

first_state = current_state['possible_next_states'][0]
return (
    current_state['optimal'],
    first_state['ins_var'],
    first_state['out_var'],
    first_state['matrix'],

```

```

        first_state['basic_vars'],
        history
    )

def update_previous_states(state):
    """Αναδρομική ενημέρωση προηγούμενων καταστάσεων"""
    if state['previous'] is not None:
        # Ενημέρωση όλων των πιθανών επόμενων καταστάσεων της προηγούμενης
        for next_state in state['previous']['possible_next_states']:
            if np.all(next_state['matrix'][0, :-1] <= 1e-8):
                next_state['optimal'] = True
            # Αναδρομική κλήση για την προηγούμενη κατάσταση
            update_previous_states(state['previous'])

        current_state['possible_next_states'].append(next_state)
        steps[step+1].append([new_basic.copy(), current_state['optimal']])
        history.append(current_state)

    first_state = current_state['possible_next_states'][0]
    return (first_state['ins_var'],
            first_state['out_var'],
            first_state['matrix'],
            first_state['basic_vars'],
            history, steps)

def run_simplex_with_optimal_propagation(initial_matrix, initial_basic):
    history = []
    current_matrix = initial_matrix.copy()
    current_basic = initial_basic.copy()
    previous_state = None

    while True:
        optimal, entering, leaving, current_matrix, current_basic, history = \
            simplex_with_history(current_matrix, current_basic, history, previous_state)

        if optimal:
            print("Βρέθηκε βέλτιστη λύση με διαδρομή:")
            print_optimal_path(history[-1])

```

```

        break
    elif entering == -2:
        print("Το πρόβλημα είναι μη φραγμένο")
        break

    previous_state = history[-1] # Ενημέρωση της προηγούμενης κατάστασης

def print_optimal_path(optimal_state):
    """Εκτυπώνει τη διαδρομή που οδήγησε σε βέλτιστη λύση"""
    path = []
    current = optimal_state

    while current is not None:
        if 'ins_var' in current and 'out_var' in current:
            path.append((current['ins_var'], current['out_var']))
            current = current.get('previous', None)

    path.reverse() # Αναστροφή για να ξεκινάει από την αρχή

    print("Διαδρομή:")
    for i, (entering, leaving) in enumerate(path, 1):
        print(f"Βήμα {i}: Εισ/η x{entering+1}, Εξ/η x{leaving+1}")

vars_indexes = np.linspace(0, 6, 7, dtype='int')
basic_vars_indexes = np.linspace(4, 6, 3, dtype='int')

BN = np.array([[1, 2, 4, -1, 1, 0, 0],
               [2, 3, -1, 1, 0, 1, 0],
               [1, 0, 1, 1, 0, 0, 1]])
b = np.array([[6], [12], [2]])
c = np.array([[2, 1, 6, -4, 0, 0, 0]])
Z = np.array([[0]])
cZ = np.hstack((c, Z))
BNb = np.hstack((BN, b))
simplex_Matrix = np.vstack((cZ, BNb))
simplex_Matrix = np.array(simplex_Matrix, dtype='float64')
history = []
run_simplex_with_optimal_propagation(simplex_Matrix, basic_vars_indexes)

```